Syntax-Aware Detection of Training Data Contamination in Code LLMs

Jiazheng Zhang[1,3], Cheng Liu[2]*, Long Cheng[2], Xiaowei Li[1,3], Huawei Li[1,3]
[1]SKLP, Institute of Computing Technology, Chinese Academy of Sciences
[2]School of Control and Computer Engineering, North China Electric Power University
[3]University of Chinese Academy of Sciences
zhangjiazheng24@mails.ucas.ac.cn, liucheng@ncepu.edu.cn

Abstract

Despite rapid advances in Code LLMs, their training datasets are often opaque due to proprietary and competitive concerns. Since many evaluation benchmarks are open-source and widely available online, models may unintentionally train on them, leading to data contamination that compromises evaluation validity. Reliable techniques for detecting whether a code snippet appears in a model's training data are therefore essential. Compared with natural language, code contamination detection presents two unique challenges. First, code tokens from different syntactic categories contribute unequally to contamination signals, making uniform token treatment ineffective. Second, models trained on semantically equivalent code variants can evade detectors based solely on text matching. To address these challenges, we propose a syntax-aware framework that explicitly models token-type contributions. With a lightweight supervised detector trained on few labels, it captures contamination signals efficiently. Experiments across 8 Code LLMs, 2 languages, and 4 code transformations show absolute AUC gains of 8–10% for pretraining data detection and 15–20% for fine-tuning data detection, with AUC exceeding 90% on most fine-tuned tasks. Our work demonstrates that syntax-aware modeling is essential for trustworthy Code LLM evaluation.

1 Introduction

The rapid advancement of Code LLMs has significantly transformed software development, enabling powerful code generation, completion, and understanding (Roziere et al., 2023; Guo et al., 2024; Zhong et al., 2025). To systematically evaluate these capabilities, various benchmarks and evaluation protocols have been proposed (Chen et al., 2021; Austin et al., 2021; Hendrycks et al., 2021; Jain et al., 2025), providing standardized metrics for model comparison. These benchmarks guide model selection and drive progress by revealing the strengths and limitations of existing approaches. (Zheng et al., 2023; Jimenez et al., 2024).

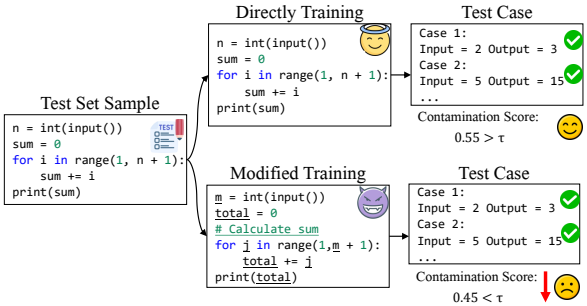


Figure 1: An example shows that rewriting test-code tokens of different syntactic types while preserving functionality allows an LLM to achieve comparable performance yet evade existing contamination detectors.

However, the reliability of these benchmarks has been increasingly called into question (Deng et al., 2024; Zhang et al., 2024a). Modern Code LLMs are trained on massive corpora sourced from public repositories, forums, and other large-scale code collections. As training datasets continue to expand, which often incorporate proprietary or undisclosed data, it becomes difficult to guarantee that benchmark test sets are fully excluded, leading to training data contamination (Xu et al., 2024; Cheng et al., 2025). Consequently, models may have been exposed to test examples during training, inflating performance and undermining the validity of empirical comparisons, which can mislead both researchers and practitioners (Chen et al., 2025).

Detecting training data contamination in language models has attracted growing attention, with prior approaches primarily relying on analyzing model behaviors such as likelihood estimation or text completion patterns (Shi et al., 2023; Zhang et al., 2025). While these methods have shown promise for natural language models, directly applying them to Code LLMs is far from trivial. Unlike natural language, source code exhibits strong

*Corresponding author.

structural regularity, where tokens of different syntactic types play inherently distinct roles in contamination detection. Treating all tokens uniformly thus introduces substantial noise, making it difficult to distinguish genuine memorization from generalization. (Koohestani et al., 2026; Dikici et al., 2026; Li et al., 2026). Furthermore, code data often contains semantically equivalent implementations that differ syntactically (e.g., variations in variable naming or code structure), which can obscure contamination signals and further degrade detection accuracy(Yang et al., 2025). A typical example can be found in Figure 1.

In this paper, we argue that a key limitation of existing approaches lies in their insufficient consideration of the intrinsic characteristics of code. Unlike prior methods that either treat all tokens equally or prune syntax-constrained tokens using hand-crafted heuristics, we propose a data-driven syntax-aware framework that dynamically models the heterogeneous contributions of different syntactic types. Specifically, we align LLM tokens with syntactic types using a two-pointer LCS-based algorithm and train a lightweight LightGBM classifier on a small set of labeled samples to estimate their relative importance. This design improves contamination detection accuracy while providing interpretable insights into token contributions. Our analysis further reveals that these contributions are highly non-uniform: certain syntactic tokens, notably brackets and specific operators, are as informative as identifiers, whereas many other keywords and operators contribute relatively little. Overall, by dynamically modeling the importance of different syntactic types, our method effectively captures contamination signals while remaining efficient and practical.

We conduct extensive experiments across 8 open-source LLMs, 2 programming languages, and 4 types of code transformations. Our approach substantially outperforms prior contamination detection methods: for pre-trained base models data detection, it achieves an AUC of approximately 75%, yielding an absolute gain of 8–10% over the best baseline (approximately 65%); for fine-tuned models data detection, the average absolute AUC improvement reaches 15–20%, with most tasks exceeding 90%. These results underscore the importance of incorporating code-specific inductive biases to ensure reliable evaluation, and our framework readily extends to Python, C++, and other programming languages. Our code and data are publicly available on GitHub.[1]

Our contributions are summarized as follows:

- We identify and analyze unique challenges of detecting contamination in Code LLMs, notably token homogeneity and semantic equivalence.
- We propose a syntax-aware detection framework that differentiates the contributions of code token categories. With a lightweight supervised model, it achieves high accuracy with only a small number of labeled samples.
- We propose a data construction pipeline for contamination detection benchmarking in Code LLMs, spanning multiple models, programming languages, and semantically equivalent transformations, and demonstrate substantial improvements over existing methods.

2 Related Work

2.1 Code LLMs and Evaluation Reliability

Code LLMs have rapidly advanced across code generation and related tasks, achieving strong performance on widely adopted benchmarks (Roziere et al., 2023; Guo et al., 2024; Gong et al., 2026; Zhang et al., 2026). These benchmarks provide a unified testing ground, yet their reliability is increasingly threatened by training data contamination, as models are trained on massive, opaque corpora scraped from public repositories, making it hard to guarantee that test samples are fully excluded (Matton et al., 2024; Li et al., 2025a).

This issue goes beyond exact duplication. Yang et al. (2025) show that training on semantically equivalent code—variants with renamed variables or restructured control flow—still inflates benchmark performance, suggesting reliance on memorization rather than reasoning. These results reveal a critical gap: existing frameworks lack mechanisms to detect whether a test sample or its semantic variants were encountered during training, underscoring the need for methods that capture semantic-level contamination.

2.2 Data Contamination Detection

General methods detect contamination via model behavior differences, using perplexity (Carlini et al., 2021) and refined scoring: Min-K% (Shi et al., 2023), Min-K%++ (Zhang et al., 2025), DC-

[1] https://github.com/zjz1222/codellm_contamination_detection

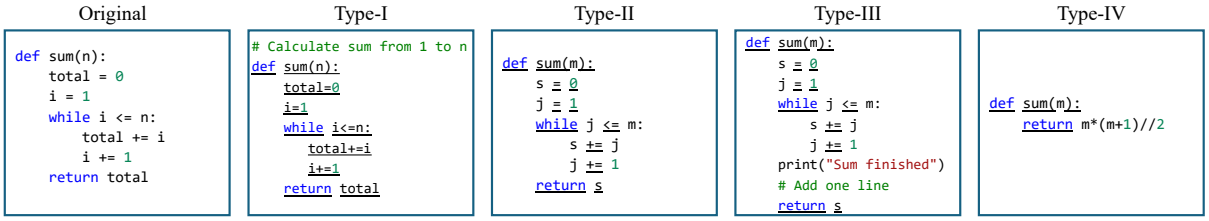


Figure 2: Examples of different types of code clones for the 1-to-n summation function, where underscored tokens denote those retained from the original code in the cloned code.

PDD (Zhang et al., 2024c), Recall (Xie et al., 2024), PETAL (He et al., 2025), Tag&Tab (Antebi et al., 2025), Infilling Score (Raoof et al., 2025). For code, Gotcha (Yang et al., 2024), CodeMI (Wan et al., 2026), Buzzier (Zhang et al., 2024b) employ shadow models. SYNPrune (Li et al., 2026) detects code contamination by excluding syntactically obligatory tokens, and Adversarial-Prompt (Jiang et al., 2025) injects semantics-preserving perturbations. However, these methods overlook token heterogeneity, fail on semantically equivalent clones, and incur high overhead from shadow models or multiple prompt completions. We propose a syntax-aware detector that distinguishes token roles via programming language semantics and learns from few labeled samples, enabling efficient detection of contamination, including semantic clones.

3 The Proposed Code Contamination Detection Framework

3.1 Problem Formulation and Key Observations

Given a target code snippet x, a large language model (LLM) $\mathcal{M}$, and its training corpus $\mathcal{D}$, the goal of data contamination detection is to determine whether x, has been included in $\mathcal{D}$. Conventional approaches rely on exact-match contamination detection, defined as:

$$A(x) = 1 \iff x \in \mathcal{D} \tag{1}$$

However, this formulation is insufficient for the code domain, where semantically equivalent implementations can exhibit substantial syntactic and structural variation. A model may therefore have been exposed to a variant of x during training even if x itself does not explicitly appear in $\mathcal{D}$. To capture such cases, we extend the task to semantic-level contamination detection:

$$A(x) = 1 \iff \exists x' \in \mathcal{D} \text{ s.t. } \mathrm{SE}(x, x') \tag{2}$$

where $\mathrm{SE}(x, x')$ denotes semantic equivalence between two code snippets.

Unlike text-domain contamination detection, which mainly considers limited surface-form perturbations, code contamination involves broader and more diverse semantics-preserving transformations. We derive our definition of code contamination detection from the classic taxonomy of code clones (Baxter et al., 1998; Roy and Cordy, 2007). Following prior work on contamination detection via semantic clones (Li et al., 2025a), we focus on Type-I to Type-III clones, which preserve semantic equivalence under varying degrees of syntactic and structural variation. Type-IV clones are excluded because they are structurally distinct despite their semantic similarity.

Figure 2 illustrates contamination under different code transformations, revealing markedly different token-level stability under semantic-preserving changes:

- **Original**: Code fragments are exactly identical, with no modifications.
- **Type-I**: Code fragments differ only in whitespace, layout, and comments, while all other tokens remain identical.
- **Type-II**: Differences extend to identifier names, with program structure preserved and keywords, operators, and constants highly stable, but identifier-related tokens begin to diverge.
- **Type-III**: Statement-level edits such as insertions, deletions, or rephrasing primarily affect identifiers, strings, and syntax, yet core computational patterns stay relatively stable.
- **Type-IV**: Code fragments are syntactically completely different but functionally identical, and we do not consider this case as data contamination.

This progression reveals that semantically critical tokens, such as operators, and structural tokens, including delimiters such as parentheses, are more invariant, while less critical tokens, such as identifiers and whitespace, concentrate the variation.

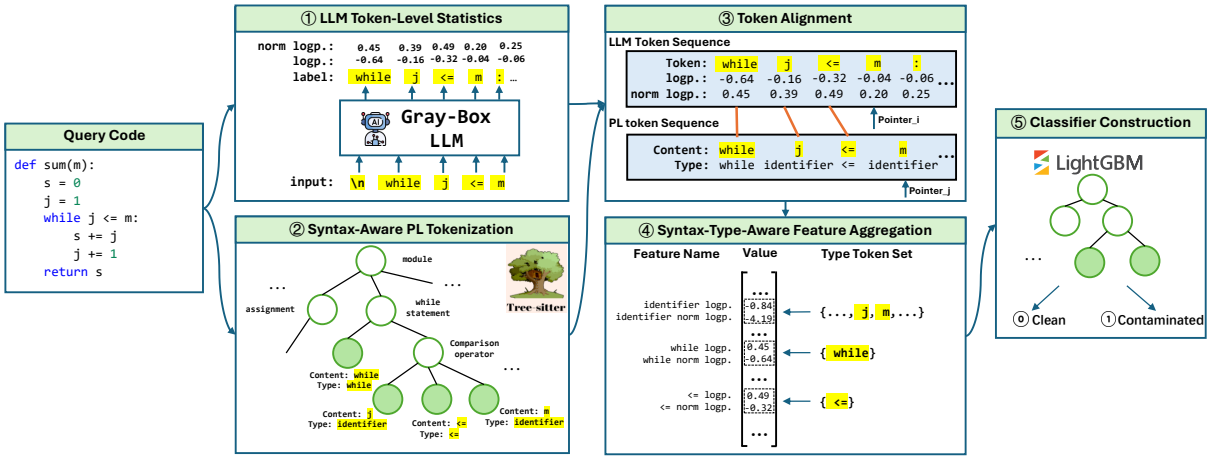


Figure 3: Overview of The Proposed Code Contamination Detection Framework

Consequently, different token categories contribute unevenly to contamination signals. Existing methods treat code as plain text with uniform token importance, introducing noise, especially with semantic clones. We thus propose a syntax-aware detection paradigm that explicitly models syntactic-type heterogeneity. By dynamically weighting tokens according to their syntactic categories, our approach is better aligned with the structural characteristics of code and enables more accurate detection of semantic-level contamination.

3.2 Framework Overview

Building upon the token-level heterogeneity observed in Section 3.2, we propose a syntax-aware data contamination detection framework, as illustrated in Figure 3. The core idea is to jointly leverage model behavior signals and code structural information, integrating them into a unified, syntax-type-aware representation that effectively distinguishes whether a code sample, or its semantic variants, has been included in the training corpus. In line with TokDrift (Li et al., 2025b), we refer to the token sequences produced by the LLM's subword tokenizer as *LLM tokens*, and to the syntax-aware tokens obtained through compiler-level parsing as *PL tokens* (where PL stands for programming language).

Specifically, we first collect a small set of training samples from the target LLM and a small set of non-training samples based on timestamps. For each given code snippet, our framework performs the following steps:

① **LLM token-level statistics.** We obtain subword token-level generation statistics via *LLM tokenization* We extract the sequence of *LLM tokens* together with their log probabilities and normalized log probabilities.

② **Syntax-aware PL tokenization.** We parse the code into *PL tokens* with their syntactic types (e.g., keyword, identifier, operator) via Tree-sitter, as these syntactic roles contribute unequally to contamination detection.

③ **Token alignment.** Since *LLM tokens* (determined by corpus frequency) and *PL tokens* (governed by syntactic constraints) differ in granularity, we design a double-pointer alignment algorithm mapping each *LLM token* to its most likely syntactic type.

④ **Syntax-type-aware feature aggregation.** For each syntactic type, we aggregate token-level statistics via the Min-$k\%$ average log probability and normalized log probability, which emphasize low-confidence tokens that are more indicative of contamination.

⑤ **Classifier construction.** Using the resulting feature vectors and the small labeled dataset, we train a lightweight classifier based on LightGBM. For a new code snippet, the classifier achieves more accurate contamination detection by dynamically accounting for the importance of different syntactic types.

3.3 LLM Token-Level Statistics

Given an LLM $\mathcal{M}$ and a code snippet x, we tokenize x into a sequence of length n:

$$x = x_0 x_1 x_2 \dots x_{n-1} \tag{3}$$

As LLMs are autoregressive, we calculate the log conditional probability for each token x_i ($1 \leq i < n$):

$$\ell(x_i) = \log p(x_i \mid x_{<i}). \tag{4}$$

Inspired by Min-k%++(Zhang et al., 2025), we consider not only the absolute probability of each target token but also its relative probability across the full vocabulary. Concretely, even if the absolute probability of a token is low, we still recognize it as a potential result of maximum likelihood training if it holds a high relative probability against other tokens in the vocabulary. We normalize the token's probability over the vocabulary with the following formula:

$$\hat{\ell}(x_i) = \frac{\log p(x_i \mid x_{<i}) - \mu(\cdot \mid x_{<i})}{\sigma(\cdot \mid x_{<i})} \quad (5)$$

where $\mu(\cdot \mid x_{<i}) = \mathbb{E}_{z \sim p(\cdot|x_{<i})}[\log p(z \mid x_{<i})]$ is the expectation of the next token's log probability over the model's vocabulary given the prefix $x_{<i}$, and $\sigma = \sqrt{\mathbb{E}_{z \sim p(\cdot|x_{<i})}[(\log p(z \mid x_{<i}) - \mu)^2]}$

Additionally, we store the end character offset of each *LLM token* produced by the tokenizer, defined as $\text{offset}(x_i) = e_i$, where e_i is the ending character position of x_i. This offset enables precise alignment with *PL tokens*. In this way, for each token x_i we obtain a 4-tuple X_i:

$$X_i = \{x_i,\ \ell(x_i),\ \hat{\ell}(x_i),\ \text{offset}(x_i)\} \quad (6)$$

that supports subsequent matching. Notably, since $\ell(x_0)$ and $\hat{\ell}(x_0)$ cannot be computed for the first token x_0, we mark it with a special value for the following token alignment process.

3.4 Syntax-Aware PL Tokenization

In this phase, we parse the raw code into an AST via Tree-sitter (Brunsfeld et al., 2026). We focus exclusively on leaf nodes, as they correspond to terminal symbols that carry concrete textual content such as variables, keywords, and literals, while non-leaf nodes only represent structural rules without surface text. For each leaf node, we record its text y_i, its syntactic type $\text{type}(y_i)$, and its end character offset $\text{offset}(y_i)$, i.e., the ending character position of y_i in the original code string. These leaf nodes collectively form the sequence of *PL tokens*, each represented by a 3-tuple

$$Y_i = \big(y_i,\ \text{type}(y_i),\ \text{offset}(y_i)\big), \quad (7)$$

which enables direct alignment with *LLM tokens*. Since we leverage Tree-sitter's standard grammars, the approach naturally generalizes across languages. For Python and C++, the PL token types extracted by Tree-sitter are grouped into 19 and 26 main categories, respectively, covering 94 and 144 distinct types, as detailed in Appendix A.1.

Algorithm 1: Token Alignment

Input: LLM tokens $X = \{X_0, \ldots, X_{n-1}\}$, Parse triplets $Y = \{Y_0, \ldots, Y_{m-1}\}$
Output: Aligned token-type pairs S

$i \leftarrow 0, j \leftarrow 0, S \leftarrow \emptyset$;
while $i < n$ **do**
  $max_lcs \leftarrow -1, \text{type}(x_i) \leftarrow \text{null}$;
  while $j < m$ ***and*** $\text{offset}(y_j) \leq \text{offset}(x_i)$ **do**
    if $\text{LCS}(x_i, y_j) > max_lcs$ **then**
      $max_lcs \leftarrow \text{LCS}(x_i, y_j)$;
      $\text{type}(x_i) \leftarrow \text{type}(y_j)$;
    $j \leftarrow j + 1$;
  $S \leftarrow S \cup \{(x_i, \ell(x_i), \hat{\ell}(x_i), \text{type}(x_i))\}$;
  $i \leftarrow i + 1$;
return S;

3.5 Token Alignment

A primary challenge is the misalignment between the *LLM token* x_i generated by the LLM and the *PL token* y_i parsed by Tree-sitter. While y_i represents a complete and atomic syntactic unit, x_i may consist of partial syntactic units or a combination of whitespace and fragments. To address this, we must determine the potential syntactic type for each token through a matching process. Leveraging the triplets from Step 1 and Step 2, we implement a two-pointer algorithm to perform this alignment, as shown in Algorithn 1. where $\text{LCS}(x_i, y_j)$ denotes the length of the Longest Common Substring(LCS) between x_i and y_j.

The core of this matching strategy involves identifying all *PL tokens* y_j proximal to each *LLM token* x_i and selecting the *PL token* that maximizes the LCS length with x_i. The syntactic type of the selected y_j is then assigned to x_i. In cases where multiple *PL tokens* yield the same maximum LCS (e.g. a token "):" matching distinct nodes ")" and ":"), we prioritize the leftmost non-whitespace token, assuming that the earliest syntactic unit determines the token's type in left-to-right generation. The resulting set S contains, per token x_i, its log probability $\ell(x_i)$, normalized log probability $\hat{\ell}(x_i)$, and assigned type $\text{type}(x_i)$. Full parsing results for Figure 3 are given in Appendix A.2.

3.6 Syntax-Type-Aware Feature Aggregation

In this phase, we transform the aligned set S into feature vectors suitable for classification. Specifi-

cally, for each syntactic type t_i, we aggregate all associated *LLM tokens* into a subset T_i, defined as:

$$T_i = \{(x_j, \ell(x_j), \hat{\ell}(x_j)) \mid \text{type}(x_j) = t_i\} \quad (8)$$

Following Shi et al. (2023), we calculate the average log probability of the MIN-$k\%$ tokens for each syntactic type t_i in a predefined syntactic type set T_i. This focus on low-confidence tokens is motivated by the observation that they are more indicative of a sample's contaminated status, providing a stronger signal for whether the data was seen during training. Let K_i and $\hat{K}_i$ be the subsets of T_i containing the tokens with the $k\%$ smallest log probabilities $\ell(x_j)$ and normalized log probabilities $\hat{\ell}(x_j)$, respectively, with the percentage rounded up. The aggregated feature $\ell(t_i)$ is computed as:

$$\ell(t_i) = \frac{1}{|K_i|} \sum_{x_j \in K_i} \ell(x_j) \quad (9)$$

And $\hat{\ell}(t_i)$ is computed as:

$$\hat{\ell}(t_i) = \frac{1}{|\hat{K}_i|} \sum_{x_j \in \hat{K}_i} \hat{\ell}(x_j) \quad (10)$$

Notably, certain syntactic types, such as "<=", may be absent from a given code snippet. For these missing types, we assign a value of `NaN` to their corresponding features $\ell(t_i)$ and $\hat{\ell}(t_i)$.

3.7 Classifier Construction

After constructing the feature vectors, we employ LightGBM (Ke et al., 2017) to train the classifier, with the following motivations:

- **Suitability for high-sparsity, limited-data, and nonlinear scenarios**. Code features are highly sparse due to frequently missing syntactic and operator types. As a tree-based ensemble, LightGBM naturally models nonlinear token–contamination relationships and performs robustly with limited samples under such sparsity.
- **Lightweight and efficient inference**. LightGBM offers fast training and inference, satisfying the strict efficiency demands of large-scale training data detection.

For each target LLM, we construct a small set of contaminated and clean samples to train a lightweight LightGBM classifier for contamination detection. Following prior work (Shi et al., 2023; Li et al., 2025a), we use publicly disclosed training data of open-source LLMs as contaminated samples and code released after the corresponding training-data cutoff as clean samples. To mitigate temporal artifacts, code-style biases, and other spurious correlations, we further apply an iterative adversarial sampling strategy over large-scale training corpora and GitHub repositories. This process combines heuristic constraints (e.g., code length and complexity distributions) with a FastText classifier to align the contaminated and clean pools in both surface-level and syntax-aware distributions. We then use Qwen-Coder-Next to generate four categories of semantically consistent variants. Finally, for each LLM, the resulting samples are split into a classifier training set for LightGBM and a detection evaluation set shared by all methods. Details of the LightGBM training procedure are provided in Appendix A.3, while the complete data construction pipeline and resulting datasets are described in Appendix B.

4 Experiments

4.1 Experimental Setup

In this section, we detail the datasets, baselines, evaluation metrics, and implementation specifics.

Models and Dataset. Following the pipeline described in Section 3.4, we construct our datasets for C++ and Python. For pre-training data detection, we employ *starcoderbase* and *starcoderbase-7b* (Li et al., 2023), while for fine-tuned model data detection, we evaluate *Magicoder-DS-6.7B*, *Magicoder-CL-7B* , *octogeex-6.7b*, *octocoder*, *OpenCoder-1.5B-Instruct*, and *OpenCoder-8B-Instruct* (Wei et al., 2023; Muennighoff et al., 2023; Huang et al., 2025). Although the evaluation datasets differ across LLMs, for each programming language we use 1,000 samples as a classifier training set for LightGBM and 4,000 samples as a shared detection test set for all methods. The test set is equally distributed across four categories: Original, Type-I, Type-II, and Type-III. In total, our experiments encompass 8 models, 2 programming languages, and 4 code transformation techniques.

Baselines. We consider the following methods as our baselines, each predicting whether an example was a member based on: (i) *PPL*: The perplexity of the example. (ii) *Zlib*: The ratio of the example's perplexity against its zlib entropy. (iii) *Min-K% Prob* (Shi et al., 2023): The average log-likelihood of the k% of tokens with the lowest prob-

Table 1: Detection performance comparison across languages and transformation types based on starcoderbase

Lang	Method	Original			Type-I			Type-II			Type-III			Macro Average		
		AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%
Python	PPL	60.0	58.1	1.8	58.2	56.5	1.6	57.2	56.2	**2.2**	58.5	57.1	1.8	58.5	57.0	1.8
	zlib	47.2	50.4	0.6	47.8	50.8	0.4	47.1	50.7	0.2	47.3	51.2	0.0	47.3	50.8	0.3
	Min-10%	61.6	59.3	1.6	60.0	58.0	1.2	58.7	57.5	2.0	60.0	59.0	1.6	60.1	58.5	1.6
	Min-20%	60.7	58.3	1.8	58.9	56.5	1.6	57.9	57.1	2.2	59.1	58.1	1.8	59.1	57.5	1.8
	Min-10%++	56.8	57.9	1.4	57.2	57.3	1.4	54.4	55.0	1.6	57.1	56.5	2.0	56.4	56.7	1.6
	Min-20%++	57.0	57.6	1.4	57.3	57.8	1.4	54.9	55.2	1.4	57.5	57.3	2.0	56.7	57.0	1.6
	Recall	67.2	63.5	**3.0**	63.4	60.6	0.8	60.9	59.3	0.6	59.7	58.2	1.0	62.8	60.4	1.4
	Synprune	67.0	63.4	1.8	65.4	62.5	**2.2**	64.9	63.7	2.0	67.1	64.1	1.2	66.1	63.4	1.8
	Ours	**76.5**	**70.7**	1.6	**74.6**	**68.2**	1.9	**74.6**	**68.4**	1.4	**73.6**	**68.4**	**2.4**	**74.8**	**68.3**	**2.0**
C++	PPL	53.1	54.4	3.4	51.1	53.8	**4.4**	50.7	53.4	3.6	49.5	53.3	3.4	51.1	53.7	3.7
	zlib	38.6	50.5	1.2	39.1	50.8	2.2	39.2	50.6	1.8	38.1	50.3	1.2	38.7	50.6	1.6
	Min-10%	55.2	55.8	3.4	53.4	53.9	4.2	52.2	53.7	3.6	50.9	53.5	2.8	52.9	54.2	3.5
	Min-20%	53.9	54.6	3.4	52.0	53.7	4.4	51.2	53.7	3.6	49.9	53.0	3.4	51.8	53.8	3.7
	Min-10%++	53.7	53.9	2.4	55.3	55.3	3.4	51.5	53.6	**4.8**	51.2	53.8	3.2	52.9	54.1	3.5
	Min-20%++	53.4	54.0	2.4	54.3	54.8	3.2	51.0	53.4	4.8	50.4	53.9	3.2	52.3	54.0	3.4
	Recall	68.4	63.9	**7.0**	67.9	64.3	4.4	66.6	62.3	1.6	64.8	61.6	2.4	66.9	63.0	**3.9**
	Ours	**73.7**	**69.2**	6.8	**72.5**	**67.4**	2.6	**72.3**	**67.1**	2.9	**71.9**	**67.1**	**4.8**	**72.8**	**67.6**	2.6

abilities. (iv) *Min-K%++ Prob* (Zhang et al., 2025): The average normalized log-likelihood of the k% of tokens with the lowest normalized probabilities, where the normalization is based on the statistics of categorical distribution over the entire vocabulary. (v) *Recall* (Xie et al., 2024): Measuring the relative drop in conditional log-likelihood triggered by clean prefixes. (vi) *Synprune* (Li et al., 2026): Prunes syntactically required tokens via manually designed rules, improving training data detection for Python only. For more details on the baselines, please refer to Appendix C.1.

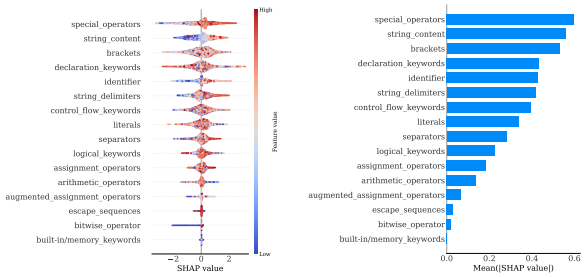
Evaluation Metrics. Following most existing works (Shi et al., 2023; Zhang et al., 2025), we evaluate using the area under the ROC curve (AUC), Balanced Accuracy (ACC), and TPR@1%FPR, i.e., the true positive rate at a 1% false positive rate. For more details, please refer to Appendix C.2.

Implementation Details. We run all Code LLM inference on two NVIDIA H100 GPUs. Specifically, we set the maximum context length to 8K tokens per LLMs, falling back to 4K if the native limit is insufficient. For the Min-K% and Min-K%++ baselines, we set k to 10 and 20. For RECALL, we use a single clean code snippet as the prefix, since random sampling from 1–12 candidates performed best. In our implementation, we set $k = 10$ and employ LightGBM as the classifier with GBDT boosting and AUC as the primary optimization objective. Detailed experimental configuration are provided in Appendix C.3.

4.2 Main Results

Table 1 summarizes the results for starcoderbase, while those for the other seven models are provided in Appendix C.4. Overall, our method consistently outperforms all baselines across models and settings. On pre-trained *StarCoder* models, most baselines perform near-randomly, with AUC 0.5–0.6 and TPR@1%FPR 0–5%, while Recall and Synprune reach 0.6–0.7 AUC. In contrast, our method achieves AUC 0.7–0.8 and Accuracy 0.65–0.75, a relative gain of 5–15%. Notably, the advantage widens on fine-tuned models: on *Magicoder* and *OpenCoder*, we attain AUC 0.9–0.95, Accuracy above 0.8, and TPR@1%FPR over 20%, against baseline AUCs around 0.7–0.8. On *OctoPack*, our AUC/Accuracy of 0.75–0.9 represents an improvement of roughly 15% over baselines' 0.4–0.7. These results indicate that detecting pre-training data is intrinsically more challenging than detecting fine-tuning data, and that smaller models from the same family are harder to detect than larger ones, likely due to differences in training strategies and model capacity.

Impact of different languages. Baselines generally perform better on Python than C++, with AUC/Accuracy drops of 0.05–0.1 and a slight TPR@1%FPR decline. Due to random prefix selection, Recall occasionally favors C++ on a few models, such as *OpenCoder*. Our method maintains strong performance on both languages: AUC 0.7–0.8 on the *StarCoder* and *OctoPack* series, with C++ slightly lower, and >0.9 on the *Magicoder* and



(a) Beeswarm (b) Feature Importance

Figure 4: Impact of Python syntactic types on detection results, averaged over eight LightGBM classifiers.

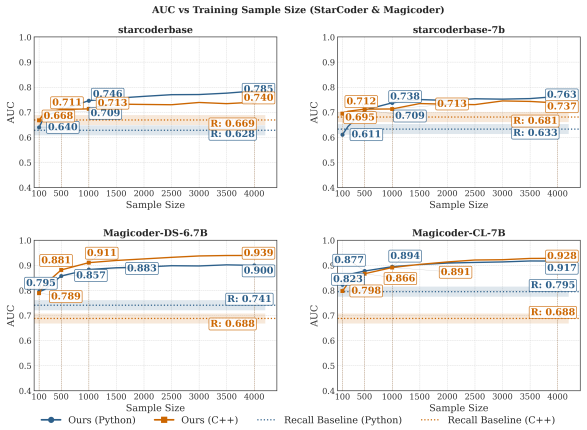


Figure 5: Impact of training data size on detection AUC for classifier, under Starcoder and Magicoder

OpenCoder series with comparable results across languages. The C++ degradation can be attributed to more complex syntax, lower sample quality, and higher learning difficulty, making it harder to precisely capture memorization signals.

Impact of different code contamination types. All baselines show decreasing performance with stronger code transformations. On fine-tuned data, accuracy drops 15–20% from Original to Type-III, whereas on pre-training data the decline is only 5%, as the model's shallower memorization of pre-training samples makes them less sensitive to code transformations. Anomalous improvements occur only on models with near-random baseline accuracy, where original and transformed samples are similarly and weakly memorized, thus exhibiting little differential effect. As the degree of code transformation increases, level reduces accuracy by 2–10%, with larger degradation on fine-tuned data. Our method, which weights token syntactic types, is more robust: AUC and Accuracy drop by only 1–3% from Original to Type-III on most models. It is thus effective for both exact-match detection and code variants contamination detection.

4.3 Effect of Syntactic Type

We employ SHAP (Lundberg and Lee, 2017), combined with the token taxonomy in Appendix A.1.2, to analyze the contribution of each major syntactic type to training data detection accuracy. For each of the eight models, we compute SHAP values on Python code and average them across models to obtain a robust importance ranking. Figure 4 shows the resulting averaged beeswarm and bar plots, where only features with positive mean importance are displayed. The SHAP analysis for C++ syntactic types can be found in Appendix C.5.

Importance Variation across Syntactic Types. We quantify importance by the confidence gap of syntactic type between contaminated and clean samples. Structural tokens, such as brackets, delimiters, and member-access operators, and string literals exhibit the largest gaps. Domain-specific strings are memorized verbatim in contaminated data, yielding high confidence, while clean strings remain generic and low-confidence. Brackets such as `(` and `)` and dots `.` reside at syntactic joints, because their prediction is strongly driven by local structural and stylistic patterns memorized from training, their occurrence becomes highly predictable once the context is fixed, leading to overfitting and a large confidence gap between contaminated and clean code. In contrast, keywords such as `if` and `for`, operators such as `+` and `=` are well generalized across all corpora, and mostly short, reusable identifiers are predicted in an open-class fashion, preventing low-entropy overfitting. Hence, keywords, operators, and identifiers show markedly smaller divergence and rank low in importance.

Commonalities and Differences across Python and C++. Structural tokens are highly important in both languages, as they receive higher confidence in contaminated samples while rarely being identically fixed in clean code, creating a large confidence gap. In Python, string literals and special operators achieve the highest importance (0.56 and 0.60), while arithmetic operators remain relatively low (0.13). In C++, arithmetic operators are more discriminative (0.41), whereas built-in types such as `int` and `double` saturate both samples with a minimal gap of 0.17. The `.` operator strongly separates contaminated from clean Python code because its low frequency makes high confidence a signal of memorization. In C++, the mandatory and pervasive use of `.` and `->` elevates baseline confidence in both settings, diluting their discriminative power.

Table 2: AUC comparison of surface-level and syntax-aware classifiers across LLMs

Classifier	StarCoderBase	OctoCoder	MagiCoder-DS-6.7B	OpenCoder-8B-Instruct
Min-10%	0.556	0.532	0.735	0.842
Min-10%++	0.561	0.569	0.705	0.789
FastText	0.549	0.556	0.628	0.622
Syntax-aware TF-IDF	0.509	0.548	0.539	0.547
Syntax-aware Log Prob.	0.668	0.707	0.850	**0.963**
Syntax-aware Norm. Log Prob.	0.620	0.713	0.836	0.936
Syntax-aware Combined Prob.	**0.678**	**0.752**	**0.864**	**0.963**

4.4 Effect of Training Sample Size

We evaluate AUC against the number of classifier training samples, varying up to 4K with a fixed 4K test set, for each detection task. Figure 5 shows the results for StarCoder and Magicoder. On both Python and C++ benchmarks, our method matches or surpasses the Recall baseline AUC using only 100 training samples. Detection accuracy steadily improves as the training set grows: the gain is substantial from 100 to 1,000 samples, but becomes marginal beyond 1,000. With larger training sets, our method consistently outperforms the Recall baseline on all tasks. Compared to pre-training data detection, fine-tuning data detection exhibits faster AUC growth and reaches higher accuracy with fewer samples. Pre-training data detection is inherently harder because contaminated samples are insufficiently trained, leading to slower AUC improvement and a lower upper bound. Results for other models are provided in the Appendix C.6.

4.5 Disentangling Memorization from Spurious Signals

To empirically verify that our method captures memorization signals rather than pre-cutoff-style code, we conduct a controlled experiment on representative LLMs using Python code, with 1,000 samples each for training and testing and without code transformation. We compare surface-level, syntax-distribution-based, and confidence-based representations. To ensure a strictly controlled comparison, all syntax-aware classifiers use LightGBM with identical hyperparameters (detailed in Appendiex C.3) and the same syntactic feature dimensions, while differing only in their feature values. The compared methods are as follows:

- **FastText**: A lightweight neural network-based n-gram text classifier measuring surface style, trained using the default hyperparameters (epoch=5, lr=0.1, dim=100), with the exception of setting wordNgrams=2 to capture local context.
- **Syntax-aware TF-IDF**: Using Tree-sitter to parse the code, the Term Frequency-Inverse Document Frequency (TF-IDF) of each syntactic type is computed as the feature vector to measure syntax distribution.
- **Syntax-aware Token Prob.**: Our syntax-aware methods use the absolute, normalized, and combined Min-K% token probabilities within each syntactic type as features.

As shown in Table 2, FastText and Syntax-aware TF-IDF achieve AUC scores close to random guessing (approximately 0.5–0.6), suggesting that the contaminated and clean samples are difficult to distinguish based on surface-level code style or syntactic-type distributions. This indicates that our sampling strategy substantially mitigates potential confounding factors, such as repository age and code style.

In contrast, our confidence-based methods achieve markedly higher AUC scores. Since these methods rely on the target LLM's token-level confidence rather than surface or syntactic distributional features, the results provide strong empirical evidence that the detected signal is associated with model memorization rather than superficial temporal or stylistic artifacts. Together, these results support the effectiveness of our adversarial data alignment and confidence-based syntax-aware modeling in mitigating such confounding effects.

Due to space constraints, experiments on classifier architectures selection, logp vs. normalized logp, the choice of hyperparameter K, and generalization across models are deferred to Appendix C.

5 Acknowledgement

This work is in part supported by the Strategic Priority Research Program of the Chinese Academy of Sciences, under Grant XDB0660103, National Natural Science Foundation of China under Grant 62674224, and the State Key Laboratory of Processors, Institute of Computing Technology, Chinese Academy of Sciences under Grant CLQ202601.

Limitations

Access to token-level probabilities. Our method requires token-level probabilities (confidence scores) from the target model, i.e., gray-box access. It is worth noting that this requirement is shared by many existing pretraining data detection and membership inference methods (Shi et al., 2023; Zhang et al., 2025; Xie et al., 2024). For open-weight models, such information is directly available, and for API-based models, an increasing number of providers offer log-probability outputs. Extending the approach to stricter black-box settings, where only sampled text can be observed, is a promising direction for future research.

Reliance on labeled set. Our detector requires only a small number of labeled examples. Although our experiments show that as few as 100 samples are sufficient to match or outperform a strong baseline, obtaining labeled data may still be challenging for commercial LLMs, where the ground-truth training membership is typically unavailable.

Ethical Considerations

Potential risks and dual use (A2). Our method promotes training-data transparency, yet it could be misused to infer proprietary or sensitive content in third-party models. We explicitly condemn such misuse; all released artifacts are intended solely for academic research and model transparency.

Scientific artifacts (B1–B3). We use existing datasets and models from public repositories, and all creators are properly cited. These artifacts are obtained under open-source licenses permitting research use and redistribution, and our use is consistent with their intended research purpose. Any artifacts we create are exclusively for academic research, compatible with original access conditions, and will not be used outside research contexts.

Data filtering and documentation (B4–B5). We apply heuristic filters to remove personally identifiable information (e.g., API keys) and short snippets, and manually inspect random samples for offensive content. The final dataset contains no personally identifiable information or offensive material. We document languages, sources, and filtering procedures; demographics are not applicable.

Human subjects, consent, and institutional review (D1–D4). This work does not involve human participants, crowdworkers, or annotators, and therefore entails no recruitment, payment, or participant instructions. All data are collected automatically from publicly available code repositories and HuggingFace datasets under open-source licenses that permit research use. No personal data are collected, personally identifiable information is filtered out, and no individual consent is required. Consequently, no institutional review board (IRB) approval was sought or required.

Use of AI assistants (E1). We used LLMs (e.g., ChatGPT) solely for language polishing. All suggestions were reviewed by the authors; no scientific content or novel ideas were generated by AI tools.

References

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.

Sagiv Antebi, Edan Habler, Asaf Shabtai, and Yuval Elovici. 2025. Tag&tab: Pretraining data detection in large language models using keyword-based membership inference attack. *arXiv preprint arXiv:2501.08454*, 5.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and 1 others. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Ira D Baxter, Andrew Yahin, Leonardo Moura, Marcelo Sant'Anna, and Lorraine Bier. 1998. Clone detection using abstract syntax trees. In *Proceedings. International Conference on Software Maintenance (Cat. No. 98CB36272)*, pages 368–377. IEEE.

Max Brunsfeld, Amaan Qureshi, Andrew Hlynskyi, Will Lillis, ObserverOfTime, Christian Clason, dundargoc, Phil Turnbull, Timothy Clem, Douglas Creager, Andrew Helwer, Antonin Delpeuch, Riley Bruins, Daumantas Kavolis, Michael Davis, Ika, bfredl, Tun-Anh Nguyn, Amin Ya, and 10 others. 2026. tree-sitter/tree-sitter: v0.26.9.

Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, and 1 others. 2021. Extracting training data from large language models. In *30th USENIX security symposium (USENIX Security 21)*, pages 2633–2650.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large

language models trained on code. *arXiv preprint arXiv:2107.03374*.

Simin Chen, Pranav Pusarla, and Baishakhi Ray. 2025. Dynamic benchmarking of reasoning capabilities in code large language models under data contamination. *arXiv preprint arXiv:2503.04149*.

Yuxing Cheng, Yi Chang, and Yuan Wu. 2025. A survey on data contamination for large language models. *arXiv preprint arXiv:2502.14425*.

Chunyuan Deng, Yilun Zhao, Xiangru Tang, Mark Gerstein, and Arman Cohan. 2024. Investigating data contamination in modern benchmarks for large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8706–8719, Mexico City, Mexico. Association for Computational Linguistics.

Kıvanç Kuzey Dikici, Serdar Kara, Semih Çağlar, Eray Tüzün, and Sinem Sav. 2026. Sersem: Selective entropy-weighted scoring for membership inference in code language models. *arXiv preprint arXiv:2604.01147*.

Junfeng Gong, Zhiyi Wei, Junying Chen, Cheng Liu, and Huawei Li. 2026. From large to small: Transferring cuda optimization expertise via reasoning graph. In *International Conference on Learning Representations*, volume 2026, pages 121082–121107.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yifan Wu, YK Li, and 1 others. 2024. Deepseek-coder: when the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196*.

Yu He, Boheng Li, Liu Liu, Zhongjie Ba, Wei Dong, Yiming Li, Zhan Qin, Kui Ren, and Chun Chen. 2025. Towards {Label-Only} membership inference attack against pre-trained large language models. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 1609–1628.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and 1 others. 2021. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*.

Siming Huang, Tianhao Cheng, Jason Klein Liu, Weidi Xu, Jiaran Hao, Liuyihan Song, Yang Xu, Jian Yang, Jiaheng Liu, Chenchen Zhang, and 1 others. 2025. Opencoder: The open cookbook for top-tier code large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 33167–33193.

Naman Jain, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, pages 58791–58831.

Yuan Jiang, Zehao Li, Shan Huang, Christoph Treude, Xiaohong Su, and Tiantian Wang. 2025. Effective code membership inference for code completion models via adversarial prompts. *arXiv preprint arXiv:2511.15107*.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157.

Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. 2017. Bag of tricks for efficient text classification. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pages 427–431, Valencia, Spain. Association for Computational Linguistics.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems*, 30.

Roham Koohestani, Ali Al-Kaswan, Jonathan Katzy, and Maliheh Izadi. 2026. Ast-pac: Ast-guided membership inference for code. *arXiv preprint arXiv:2602.13240*.

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, and 1 others. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*.

Tianlin Li, Yunxiang Wei, Zhiming Li, Aishan Liu, Qing Guo, Xianglong Liu, Dongning Sun, and Yang Liu. 2025a. Investigating training data detection in ai coders. *arXiv preprint arXiv:2507.17389*.

Yinxi Li, Yuntian Deng, and Pengyu Nie. 2025b. Tokdrift: When llm speaks in subwords but code speaks in grammar. *arXiv preprint arXiv:2510.14972*.

Yuanheng Li, Zhuoyang Chen, Xiaoyun Liu, Yuhao Wang, Mingwei Liu, Yang Shi, Kaifeng Huang, and Shengjie Zhao. 2026. Uncovering pretraining code in llms: A syntax-aware attribution approach. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 721–729.

Scott M Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. *Advances in neural information processing systems*, 30.

Alexandre Matton, Tom Sherborne, Dennis Aumiller, Elena Tommasone, Milad Alizadeh, Jingyi He,

Raymond Ma, Maxime Voisin, Ellen Gilsenan-McMahon, and Matthias Gallé. 2024. On leakage of code generation evaluation datasets. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 13215–13223.

Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. 2023. Octopack: Instruction tuning code large language models. *arXiv preprint arXiv:2308.07124*.

Negin Raoof, Litu Rout, Giannis Daras, Sujay Sanghavi, Constantine Caramanis, Sanjay Shakkottai, and Alex Dimakis. 2025. Infilling score: A pretraining data detection algorithm for large language models. In *The Thirteenth International Conference on Learning Representations*.

Chanchal Kumar Roy and James R Cordy. 2007. A survey on software clone detection research. *Queen's School of computing TR*, 541(115):64–68.

Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, and 1 others. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. 2023. Detecting pretraining data from large language models. *arXiv preprint arXiv:2310.16789*.

Yao Wan, Guanghua Wan, Shijie Zhang, Hongyu Zhang, Yulei Sui, Pan Zhou, Hai Jin, and Lichao Sun. 2026. Does your neural code completion model use my code? a membership inference approach. *ACM Transactions on Software Engineering and Methodology*, 35(3):1–24.

Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2023. Magicoder: Empowering code generation with oss-instruct. *arXiv preprint arXiv:2312.02120*.

Roy Xie, Junlin Wang, Ruomin Huang, Minxing Zhang, Rong Ge, Jian Pei, Neil Zhenqiang Gong, and Bhuwan Dhingra. 2024. Recall: Membership inference via relative conditional log-likelihoods. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8671–8689.

Cheng Xu, Shuhao Guan, Derek Greene, M Kechadi, and 1 others. 2024. Benchmark data contamination of large language models: A survey. *arXiv preprint arXiv:2406.04244*.

Hua Yang, Alejandro Velasco, Thanh Le-Cong, Md Nazmul Haque, Bowen Xu, and Denys Poshyvanyk. 2025. How do semantically equivalent code transformations impact membership inference on llms for code? *arXiv preprint arXiv:2512.15468*.

Zhou Yang, Zhipeng Zhao, Chenyu Wang, Jieke Shi, Dongsun Kim, DongGyun Han, and David Lo. 2024. Gotcha! this model uses my code! evaluating membership leakage risks in code models. *IEEE Transactions on Software Engineering*, 50(12):3290–3306.

Samuel Yeom, Irene Giacomelli, Matt Fredrikson, and Somesh Jha. 2018. Privacy risk in machine learning: Analyzing the connection to overfitting. In *2018 IEEE 31st computer security foundations symposium (CSF)*, pages 268–282. IEEE.

Huixuan Zhang, Yun Lin, and Xiaojun Wan. 2024a. PaCoST: Paired confidence significance testing for benchmark contamination detection in large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1794–1809, Miami, Florida, USA. Association for Computational Linguistics.

Jiazheng Zhang, Cheng Liu, Long Cheng, Xiaowei Li, and Huawei Li. 2026. Understanding and mitigating errors of llm-generated rtl code. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*.

Jingyang Zhang, Jingwei Sun, Eric Yeats, Yang Ouyang, Martin Kuo, Jianyi Zhang, Hao Frank Yang, and Hai Li. 2025. Min-k%++: Improved baseline for pre-training data detection from large language models. In *The Thirteenth International Conference on Learning Representations*.

Sheng Zhang, Hui Li, and Rongrong Ji. 2024b. Code membership inference for detecting unauthorized data use in code pre-trained language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 10593–10603.

Weichao Zhang, Ruqing Zhang, Jiafeng Guo, Maarten de Rijke, Yixing Fan, and Xueqi Cheng. 2024c. Pretraining data detection for large language models: A divergence-based calibration method. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 5263–5274.

Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, and 1 others. 2023. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*, pages 5673–5684.

Suzhen Zhong, Ying Zou, and Bram Adams. 2025. Developer-llm conversations: An empirical study of interactions and generated code quality. *arXiv preprint arXiv:2509.10402*.

Table 3: Summary of terminal token categories of Python code after Tree-Sitter parsing.

Main Category	PL Token Categories	Example
Identifier	`identifier`	`x = my_var`
Keywords		
Declaration	`def, class, lambda, async, global, nonlocal, import, from`	`def f(): pass`
Control Flow	`if, for, while, try, with, break, continue, pass, return, yield, raise, assert, await else, elif, except, finally, as`	`for x in list:`
Logical	`and, or, not, is, in`	`a and not b`
Built-in / Memory	`print, del`	`del x`
Delimiters & Punctuation		
Brackets	`( , [ , { , ) , ] , }`	`def f(a):`
Separators	`, ; :`	`a, b; c`
Operators		
Arithmetic	`+, -, *, /, //, %, **`	`a + b`
Bitwise	`&, |, ^, ~, «, »`	`flag & mask`
Augmented assignment	`+=, -=, *=, /=, //=, &=, |=, «=, ^=`	`x += 1`
Assignment	`=`	`a = 5`
Special Operators	`@, ->, .`	`@decorator, x -> y`
Literals	`integer, float, true, false, none, ellipsis`	`return 42, false`
String		
String Delimiters	`string_start, string_end`	`"hello"`
String Content	`string_content`	`"hello"`
Escape Sequences	`escape_sequence`	`"...\n"`
String Interpolation	`escape_interpolation`	`f"...{var}"`
Whitespace	`whitespace, newline, line_continuation`	`x = 1__`
Other	`block, type_conversion`	`int(x)`

A Details of the Proposed Framework

A.1 Syntax-Aware PL Tokenization Details

A.1.1 Code Parse Result Example

As illustrated in Figure 7, the parsing structure of the example code from overview framework reveals that each leaf node represents a terminal symbol. Each node encapsulates a specific syntactic type (e.g., identifier, while, <=) and its corresponding coordinates (e.g., [3,4]-[3,9], denoting the span from the 4th to the 9th character on the third line). Leveraging this fine-grained information, we can precisely map the types and textual content of PL tokens to LLM tokens, thereby facilitating token-level type inference and subsequent training code detection.

A.1.2 Main PL Token Types

In both Python and C++, Tree-sitter parsing yields a wide variety of syntactic token types for programming languages. Rather than listing them exhaustively, we group these terminal symbols into a small set of functionally motivated categories, as summarized in Tables 3 and 4.

- **Identifiers** name entities such as variables, functions, types, and namespaces. They carry the majority of semantic information and are central to representation learning.
- **Keywords** encode the syntactic skeleton of a program: declarations, control flow, logical connectives, and memory management. They define the structure within which identifiers operate.
- **Operators and assignment symbols** describe computation and state mutation. This includes arithmetic, bitwise, logical, and augmented assignment operators, which together capture the data-flow and side-effect behavior of expressions.
- **Literals** supply primitive values (numbers, booleans, characters, strings, null pointers). They are the concrete data that flow through the program.
- **Brackets, delimiters, and punctuation** determine scope, precedence, and argument boundaries. They are essential for recovering the syntactic tree structure yet rarely carry standalone semantics.
- **Whitespace and comments** are semantically

Table 4: Summary of terminal token categories of C++ code after Tree-Sitter parsing.

Main Category	PL Token Categories	Example
Identifiers	identifier, field_identifier, type_identifier, namespace_identifier, statement_identifier, primitive_type	int my_var; std::vector int x;
Keywords		
Declaration	class, struct, enum, typedef, using, namespace	class MyClass {};
Type Qualifiers	const, volatile, mutable, static, extern, signed, unsigned, long, short, auto, decltype, constexpr, inline, virtual, explicit, friend, public, private, protected, noexcept, alignof, static_assert, offsetof, restrict	const int x;
Control Flow	if, for, while, do, switch, case, default, break	for (int i=0;...)
Logical	and, or, not	a and b
Memory/Object	new, delete, this, sizeof	new int;
Other	operator, override	void operator++()
Preprocessor Directives		
Directives (all)	#if, #ifdef, #ifndef, #include, #define, #else, #elif, #endif	#include <iostream> #ifdef DEBUG . . . #endif
Other	defined, preproc_arg, preproc_directive	#if defined(WIN32)
Operators		
Arithmetic	+, -, *, /, %, ++, --	x + y
Bitwise	&, |, ^, ~, «, »	a « 2
Comparison	==, !=, <, >, <=, >=	if (x == 0)
Logical	&&, ||, !	!a && b
Assignment	=	a = 5
Augmented Assignment	+=, -=, *=, /=, %=, «=, »=, &=, |=, ^=	x += 1
Misc	., ->, ::, ?, (), []	obj.meth(); vec[]
Brackets & Delimiters		
Brackets	(,[,{, [[,) ,], } ,]]	func(a, b) [0] {}
Separators	; : ...	int x; label: void f(...)
Literals		
Numeric	number_literal, literal_suffix	return 42; 3.14f
Boolean	true, false	true
Null Pointer	nullptr, NULL	nullptr
Character	', L', character	auto c = 'A'; L'X'
String	", L", string_content, escape_sequence	auto s = "Hello"; "line\n"
Raw String	R", raw_string_content, raw_string_delimiter	R"(raw)"
System Lib String	system_lib_string	<iostream>
Whitespace & Comments	whitespace, newline, \n, comment	// comment

inert but required for lossless source representation.

- **Preprocessor directives** (C++ only) constitute a meta-level of program construction, controlling macro expansion and conditional compilation before the core language is even invoked.

Based on this categorization, we can more precisely determine, overall, which types of syntactic elements play a more important role when probing training data.

A.1.3 PL token type frequency distribution

We conduct a statistical analysis across 64K code samples in test set for each language. The frequency distribution of PL token types for Python and C++ is illustrated in Figure 8 and 9. As observed, identifier is the most predominant type, followed by structural symbols such as , and). Additionally, keywords and literals like def and integer also exhibit high frequencies.

A.2 Token-Alignment Example

We tokenize the simple Python function in Figure 3 using Magicoder-DS-6.7B; the resulting tokenization is shown in Figure 10.

The resulting sequence of `(token, type)` pairs faithfully reflects the syntactic role of every lexical unit. Notably, the keyword `"def"` is correctly mapped to `def`, the immediately following new-

```python
def sum(m):
    s = 0
    j = 1
    while j <= m:
        s += j
        j += 1
    return s
```

Figure 6: Example python code.

```
module [0, 0] - [7, 0]
  function_definition [0, 0] - [6, 12]
    def [0, 0] - [0, 3]
    name: identifier [0, 4] - [0, 7]
    parameters: parameters [0, 7] - [0, 10]
      ( [0, 7] - [0, 8]
      identifier [0, 8] - [0, 9]
      ) [0, 9] - [0, 10]
    : [0, 10] - [0, 11]
    body: block [1, 4] - [6, 12]
      expression_statement [1, 4] - [1, 9]
        assignment [1, 4] - [1, 9]
          left: identifier [1, 4] - [1, 5]
          = [1, 6] - [1, 7]
          right: integer [1, 8] - [1, 9]
      expression_statement [2, 4] - [2, 9]
        assignment [2, 4] - [2, 9]
          left: identifier [2, 4] - [2, 5]
          = [2, 6] - [2, 7]
          right: integer [2, 8] - [2, 9]
      while_statement [3, 4] - [5, 14]
        while [3, 4] - [3, 9]
        condition: comparison_operator [3, 10] - [3, 16]
          identifier [3, 10] - [3, 11]
          operators: <= [3, 12] - [3, 14]
          identifier [3, 15] - [3, 16]
        : [3, 16] - [3, 17]
        body: block [4, 8] - [5, 14]
          expression_statement [4, 8] - [4, 14]
            augmented_assignment [4, 8] - [4, 14]
              left: identifier [4, 8] - [4, 9]
              operator: += [4, 10] - [4, 12]
              right: identifier [4, 13] - [4, 14]
          expression_statement [5, 8] - [5, 14]
            augmented_assignment [5, 8] - [5, 14]
              left: identifier [5, 8] - [5, 9]
              operator: += [5, 10] - [5, 12]
              right: integer [5, 13] - [5, 14]
      return_statement [6, 4] - [6, 12]
        return [6, 4] - [6, 10]
        identifier [6, 11] - [6, 12]
```

Figure 7: Code parse result for example.

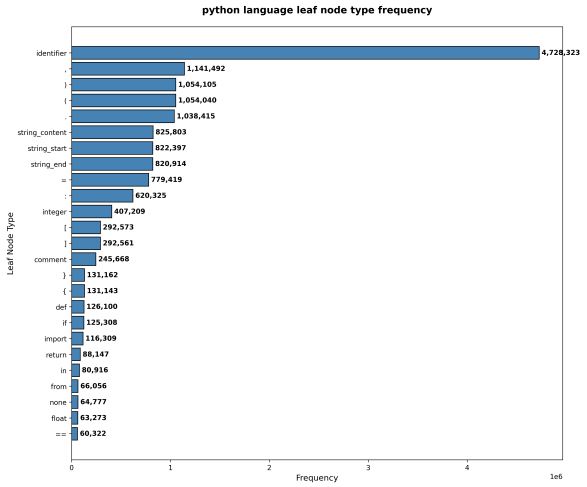


Figure 8: PL token type frequency distribution in test set based on Python language.

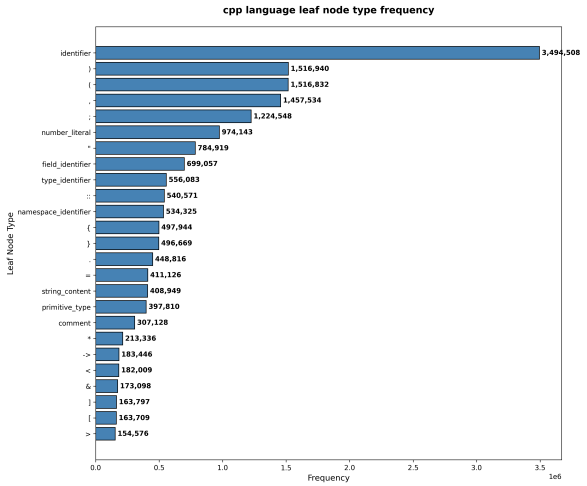


Figure 9: PL token type frequency distribution in test set based on C++ language.

line "\n" to newline, and the three-space indentation " " to whitespace. The function name " sum", which carries a leading space in the token sequence, is classified as an identifier; opening and closing parentheses "(" and ")" are marked as delimiters; the colon ":" as a delimiter; and the operators token " = ", " <=", " += " are assigned to their respective operator categories. Integer literals such as "0" and "1" are identified as integer, while all variables (" s", " j", "m") are consistently labeled as identifier. Whitespace tokens (both single spaces and indentation strings) are preserved and labeled as whitespace, and every newline is correctly recorded. Through token alignment, we can accurately capture the syntactic type associated with each LLM token, thereby enabling importance analysis of tokens of different

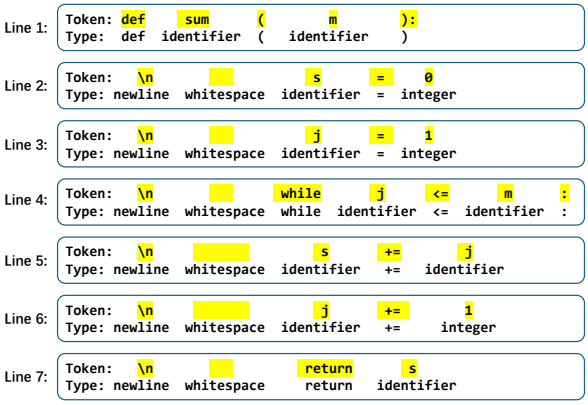


Figure 10: Tokenization of an example code snippet by Magicoder-DS-6.7B, showing each token and its associated syntactic type.

syntactic types in downstream tasks.

A.3 LightGBM Training Process

The training process of LightGBM can be summarized as follows:

1. **Model initialization.** We start with a constant base model $f_0(X)$. For regression, it is initialized as the mean of target values; for classification, it takes the majority class, denoted as $\hat{y}_0$.

2. **Gradient and Hessian computation.** For each sample, we compute the first-order gradient and second-order Hessian matrix of the loss function with respect to the current model predictions, which guide the direction and magnitude of model updates.

3. **Decision tree construction.** A new decision tree is built based on the computed gradients and Hessians, with histogram-based optimization to accelerate splitting. Nodes are split to maximize information gain or loss reduction, and pruning strategies (e.g., depth limit, minimum leaf samples) are applied to prevent overfitting.

4. **Model update.** The model is updated by adding the output of the newly trained tree scaled by a learning rate α:

$$\hat{Y}_1 = \hat{Y}_0 + \alpha f_1(X).$$

   The learning rate controls the contribution of each tree to the ensemble.

5. **Iterative ensemble.** Steps 2–4 are repeated until a predefined stopping criterion is met (e.g., maximum iterations or loss convergence). The final LightGBM model is an ensemble of multiple decision trees, whose predictions are aggregated additively.

B Dataset Construction Pipeline

In this section, we present a dataset and benchmark construction pipeline for data contamination detection in the code domain, as illustrated in Figure 11. Our approach natively supports multiple models, training phases (pretraining and fine-tuning), programming languages (Python and C++), as well as diverse semantic-preserving code transformation scenarios (i.e., code duplication, where transformed samples with unchanged semantics are still considered contaminated). It also features excellent extensibility, supporting dynamic updates and seamless extension to other models and programming languages.

B.1 Data Collection

This section details the acquisition of contaminated and clean data. All data used in this study are sourced from GitHub and HuggingFace under open-source licenses that permit use and redistribution for research purposes. For contaminated data, we select LLMs with open-source training sets: the StarCoder family is employed for pre-training data detection, while Magicoder, OctoPack, and OpenCoder series are utilized for fine-tuning stage detection. In total, we sample contaminated data from the open-source training corpora of eight models across these four distinct families. For clean data, we crawl GitHub repositories created after January 1, 2025, across various programming languages. Although a marginal overlap between these repositories and the contaminated data may exist, we posit that such instances are statistically rare and that detectors should consistently assign higher scores to genuine contaminated data. Finally, we apply heuristic filters to remove license headers, sensitive information (e.g., API keys), and snippets containing fewer than five lines of functional logic.

B.2 Style-Confusing Sampling

To prevent code style from acting as a confounding variable, we propose a style-confusing sampling strategy. A key risk in data contamination detection is that classifiers may exploit systematic style discrepancies between contaminated and clean samples, rather than genuine memorization signals, to achieve spuriously high performance. Our strategy

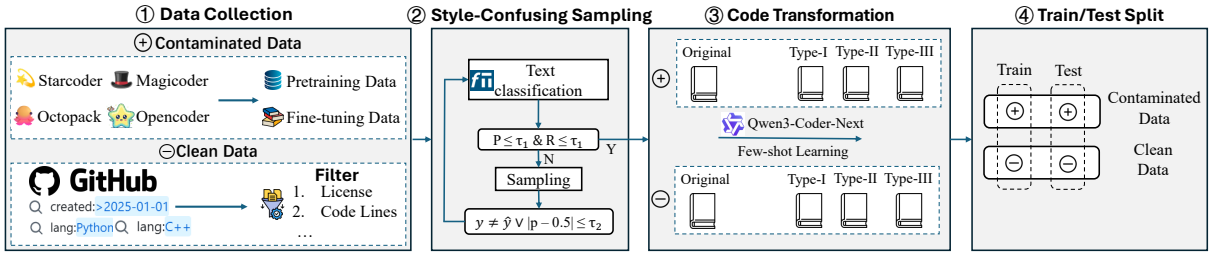


Figure 11: Overview of dataset construction pipeline.

mitigates this by iteratively filtering out samples that exhibit obvious stylistic differences, thereby forcing the detector to rely on more substantive evidence of memorization.

Concretely, given the collected contaminated and clean samples, we first train a lightweight FastText (Joulin et al., 2017) classifier to distinguish between the two sets. If both Precision and Recall fall below a threshold τ_1 (set to 0.55, close to random guessing), we consider their code styles sufficiently aligned and terminate the loop. Otherwise, we select hard-to-distinguish samples, i.e., those with low prediction confidence (satisfying $|p - 0.5| \leq \tau_2$, with $\tau_2 = 0.2$) or misclassified labels, and construct a new subset from them. The training and filtering process repeats iteratively until the termination condition is met. The complete procedure is detailed in Algorithm 2.

B.3 Code Transformation

Following the problem definition, our goal is to generate semantically equivalent code variants from an original sample. Drawing on the standard notion of code clones, we design three categories of transformations, all implemented via few-shot prompting with the Qwen3-Coder-Next model.

Type-I transformations modify only superficial elements: comments, line breaks, and whitespaces are inserted without altering any code tokens or logic. **Type-II** transformations additionally rename variables and modify constants while preserving the program structure. **Type-III** transformations further allow minor additions, deletions, and replacements of statements, as long as the semantics remain unchanged.

Figure 12 shows an example of Type-I transformations, where the original code is altered only by inserting a comment and adjusting blank lines, leaving the functionality completely identical.

Building on Type-I, Type-II transformations introduce further syntactic variation. As illustrated in Figure 13, variable names such as `sum` and `i` are changed to `total` and `num`, yet the core logic stays exactly the same.

```
You are an expert in Python programming and code analysis.
Your task is to generate a Type I clone for a given Python code snippet.

You need to refer to the example below to modify the format of the original code
by inserting or deleting whitespace, line breaks, comments, etc.

[Input]:
```python
sum = 0
for i in range(1, 6):
    sum += i
print(sum)
```
[Output]:
```python
# Calculates the sum
sum=0

for i in range(1,6):
    sum+=i

# Print sum
print(sum)
```
```

Figure 12: The Type-I prompt used for code clone generation with Qwen-3-Coder-Next.

Type-III transformations permit structural changes. In the example of Figure 14, a `for` loop is replaced by a `while` loop, while the overall functionality is preserved. This category still excludes any semantic alteration.

By incorporating these three transformation types, our data contamination detection benchmark and dataset covers both the original samples and their semantically equivalent variants, enabling a comprehensive evaluation of detection robustness across different code transformation scenarios.

B.4 Train/Test Split

Through the above procedures, we construct an expanded dataset of contaminated and clean code samples tailored for code-domain tasks. To simulate realistic data contamination detection scenarios, we split the dataset into a training set and a test set. Specifically, for each model, we randomly draw 1K samples to serve as detector training data for methods that require a training phase, and 4K samples for evaluation, where all detectors, including training-free baselines, are assessed. Both sets

Algorithm 2: Style-Confusing Sampling

Input : Member set $\mathcal{M}$, non-member set $\mathcal{N}$; AUC threshold τ_1, confidence threshold τ_2; max iterations T

Output : Hard member subset $\mathcal{M}' \subseteq \mathcal{M}$, hard non-member subset $\mathcal{N}' \subseteq \mathcal{N}$

// Initialize with full datasets

$\mathcal{M}_{\text{curr}} \leftarrow \mathcal{M}, \mathcal{N}_{\text{curr}} \leftarrow \mathcal{N}$, iter $\leftarrow 0$;

while *iter* $< T$ **do**

  iter $\leftarrow$ iter $+ 1$;

  // Terminate if empty subsets

  if $|\mathcal{M}_{curr}| = 0$ ***or*** $|\mathcal{N}_{curr}| = 0$ **then**

    break

  // Train and predict with FastText

  Train FastText on $\mathcal{M}_{\text{curr}} \cup \mathcal{N}_{\text{curr}}$;

  Predict membership probability p and label $\hat{y}$ for all samples ;

  // Terminate if discriminability vanishes

  Compute Precision and Recall on training set;

  if Precision $\leq \tau_1$ and Recall $\leq \tau_1$ **break**

  // Filter hard samples: misclassified or low-confidence

  Retain samples with $\hat{y} \neq y$ **or** $|p - 0.5| \leq \tau_2$;

  // Update subsets with hard samples

  $\mathcal{M}_{\text{curr}} \leftarrow$ filtered members, $\mathcal{N}_{\text{curr}} \leftarrow$ filtered non-members ;

return $\mathcal{M}' \leftarrow \mathcal{M}_{\text{curr}}, \mathcal{N}' \leftarrow \mathcal{N}_{\text{curr}}$

maintain an even distribution of Original, Type-I, Type-II, and Type-III code samples.

C Supplementary Experiment Details

C.1 Baseline Details

The baselines are all based on a detection score to determine a text x whether was included in the training corpus of an LLM $\mathcal{M}$.

PPL. This baseline (Carlini et al., 2021) instantiates the loss-based membership inference attack of Yeom et al. (2018) using perplexity as the loss. For language models, the loss on a sequence x is its perplexity; hence the detection score is simply the perplexity of x. A *lower* score suggests that x was part of the pretraining data.

```
You are an expert in Python programming and code analysis.
Your task is to generate a Type II clone for a given Python code snippet.

You need to refer to the example below to modify the original code
by renamed variables, changed literals etc.

[Input]:
```python
sum = 0
for i in range(1, 6):
    sum += i
print(sum)
```
[Output]:
```python
# Calculate the sum from 1 to 5
total = 0
for num in range(1, 6):
    total += num
print(total)
```
```

Figure 13: The Type-II prompt used for code clone generation with Qwen-3-Coder-Next.

```
You are an expert in Python programming and code analysis.
Your task is to generate a Type III clone for a given C++ code snippet.

You need to refer to the example below to make minor adjustments to
the logic of the original code by using different logic, algorithms,
control structures or renamed variables, but ensure the same functionality.

[Input]:
```python
sum = 0
for i in range(1, 6):
    sum += i
print(sum)
```
[Output]:
```python
# Calculate the sum from 1 to 5

sum = 0
num = 1
while num < 6:
    sum += num
    num += 1
print(sum)
```
```

Figure 14: The Type-III prompt used for code clone generation with Qwen-3-Coder-Next.

Zlib. Carlini et al. (2021) also propose to replace perplexity with the *zlib entropy* of x, i.e., the number of bits required to compress x using the zlib compression libraryThe detection score is then defined as the ratio between the model's perplexity on x and its zlib entropy. A *lower* ratio indicates that x is more likely to be a training member.

Min-K% Prob. (Shi et al., 2023) observe that non-member examples tend to contain more tokens with low probabilities. Thus, Min-K% Prob first computes the probability of each token in x, then selects the $k\%$ of tokens with the lowest probabilities, and finally outputs their average log-likelihood as the detection score. A *higher* score suggests membership in the pretraining data.

Min-$K\%$++ Prob. (Zhang et al., 2025) refine the above idea by normalizing each token probability before selection. Concretely, for each token position, the raw probability is normalized by subtracting the mean and dividing by the standard deviation of the categorical distribution over the entire vocabulary. Afterwards, the $k\%$ of tokens with the smallest *normalized* probabilities are identified, and their average normalized log-probability is taken as the detection score. Again, a *higher* score indicates that x was likely seen during pretraining.

RECALL. (Xie et al., 2024) propose RECALL (Relative Conditional LogLikelihood), a membership inference attack that measures the relative change in log-likelihood when conditioning the target sequence x on a non-member prefix P. Concretely, the detection score is defined as the ratio $\mathrm{LL}(x \mid P)/\mathrm{LL}(x)$, where $\mathrm{LL}(\cdot)$ denotes the log-likelihood assigned by the target LLM. Member data tend to exhibit a larger conditional drop, leading to a smaller numerator and thus a *higher* ratio compared to non-member data. Therefore, a higher RECALL score indicates that x was more likely used in pretraining.

SYNPRUNE. (Li et al., 2026) introduce SYNPRUNE, a syntax-pruned membership inference attack designed specifically for source code. Unlike prior methods that treat code as flat text, SYNPRUNE leverages the syntactic structure of programming languages to filter out *consequent tokens* that are deterministically required by the grammar and do not reflect authorial style (e.g., parentheses, braces, or mandatory keywords). After parsing the code, only the tokens that represent genuine stylistic choices are retained for score computation. The final detection score is the average log-likelihood over these retained tokens. A *higher* score indicates that the code snippet was likely seen during pretraining.

C.2 Metrics

We evaluate training data contamination detection methods using three standard metrics:

- **Balanced Accuracy.** The simplest efficacy measure: the proportion of correct predictions on a balanced dataset of training and non-training samples.
- **AUC.** The area under the receiver operating characteristic (ROC) curve, capturing average-case detection success. AUC is the most widely reported metric in prior work on detecting training data contamination in pre-trained LLMs.
- **TPR at Low FPR.** The true positive rate when the threshold is set to achieve a very low false positive rate. This metric directly reflects the extent of data leakage and is commonly recommended in related privacy-attack evaluations.

Existing studies largely focus on AUC, motivated by the goal of large-scale pretraining data extraction. AUC is also especially relevant in applications such as auditing machine unlearning and detecting copyright infringement.

C.3 Experimental Configuration

LightGBM Hyperparameter Settings. For the LightGBM (Ke et al., 2017) classifier, we tune hyperparameters on the main benchmark using Optuna (Akiba et al., 2019). The search space covers: number of leaves (20–150), maximum depth ($\{-1, 5, 10, 20, 30\}$), minimum data in leaf (10–100), learning rate (log-uniform in $[0.01, 0.3]$), feature fraction and bagging fraction (0.6–1.0), L1 and L2 regularization strengths (0–5), path smooth (0–10), minimum gain to split (0–10), and minimum sum of hessian in leaf (log-uniform in $[10^{-4}, 10]$). We set `max_bin`=255 and enable extra trees. Each trial trains for up to 500 boosting rounds with early stopping (patience of 50 rounds) on the validation set, combined with Optuna's pruning callback.

After tuning, we adopt a single fixed configuration across all remaining experiments to ensure stability and comparability: 50 leaves, unlimited depth, learning rate 0.1, feature fraction 1.0, bagging fraction 0.8, L1 regularization 0.2, L2 regularization 5.0, path smooth 2.0, and extra trees enabled. Other parameters (e.g., binary objective, AUC metric) are shared across all runs.

Random Seed and Result Reporting. All experiments use a fixed random seed of 42. Each experiment is repeated three times, and the reported results are averages over these three runs. The variance across runs was negligible, confirming the stability of our evaluations.

Software Dependencies. Our implementation relies on PyTorch 2.9.1, HuggingFace Transformers 4.44.2, LightGBM 4.6.0, and Optuna 4.8.0.

Data processing is performed with NumPy 1.26.0, pandas 2.3.3, PyYAML 6.0.1, and tqdm 4.67.3. All evaluation metrics are computed using scikit-learn 1.7.2, and plots are generated with matplotlib 3.10.8.

C.4 Main Results of Other LLMs

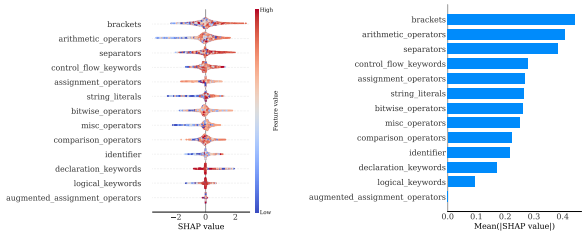
We summarize the main findings based on the seven evaluated models (Tables 5–11) as follows.

- **Overall AUC superiority.** Our method consistently achieves the highest macro-average AUC across all models, languages, and clone types. For instance, on StarCoderBase-7B it reaches 74.0% (Python) and 72.7% (C++), outperforming the strongest baseline (Recall) by 10.7 and 4.6 percentage points, respectively. The advantage grows with model capability: on OpenCoder-8B-Instruct, macro-average AUC rises to 91.5% / 92.8%, while the best competitor remains below 86%.

- **Extreme low-FPR detection (TPR@1%).** Gains in TPR@1% are even more pronounced. On OpenCoder-8B-Instruct (Python, Original), our TPR@1% reaches 61.4% versus 14.8% from the next best method (Min-20%++). Across all settings, this corresponds to a 3–6× improvement over the strongest baseline, demonstrating high precision in identifying training samples with very few false alarms.

- **Robustness to code transformation types.** Performance degrades moderately from exact copies to semantically-equivalent rewrites, yet our method remains highly robust. On Magicoder-CL-7B (Python), AUC decreases only from 91.9% (Original) to 87.8% (Type-III), and TPR@1% retains 16.6% in the hardest Type-III setting, while Recall drops to 2.6%. This indicates that the learned signal captures authorship-related information beyond surface-level token overlap.

- **Language consistency and model-dependent difficulty.** The relative ranking of methods is highly consistent between Python and C++. Detection is generally easier on larger or instruction-tuned models (e.g., OpenCoder-8B-Instruct, Magicoder-CL-7B), where token probability patterns carry richer membership information. Even on the most challenging model (OctoGeeX), where many baselines barely outperform random guessing, our method maintains an AUC above 70%, confirming strong generalization.



(a) Beeswarm (b) Feature Importance

Figure 15: Impact of C++ syntactic token types on detection results, averaged over eight LightGBM classifiers.

Overall, these results establish a new state of the art for training data membership inference, with particularly large margins in the practically critical low-false-positive regime.

C.5 Effect of C++ Syntactic Type

As shown in Figure 15, focusing on the C++ programming language, our analysis reveals that structural delimiters such as brackets, arithmetic operators, and separators are the most influential syntactic categories for data contamination detection. Control flow keywords, assignment operators, string literals, and bitwise operators form a second tier with moderate importance. In contrast, identifiers, declaration keywords, and logical keywords contribute only marginally, and augmented assignment operators are negligible. This distribution suggests that operator usage patterns and structural delimiters, capturing more locally memorized information, provide stronger evidence for detecting training data contamination than widely shared keywords or generic identifiers in C++ code.

C.6 Effect of Training Sample Size (Octocoder & OpenCoder)

Figure 16 shows the test-set AUC of our detector on the *OctoPack* and *OpenCoder* model families as a function of the number of training samples. Training our LightGBM detector with as few as 100 samples already surpasses the Recall baseline by a considerable margin. For example, on *OpenCoder-1.5B-Instruct*, the Recall baseline yields an AUC around 0.62 for both Python and C++, whereas our method trained on only 100 samples achieves 0.797 and 0.874, respectively, demonstrating strong detection accuracy even with severely limited training data. Performance further improves as more train-

Table 5: Detection performance comparison across languages and code transformation types based on starcoderbase-7b

Lang	Method	Original			Type-I			Type-II			Type-III			Macro Average		
		AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%
Python	PPL	60.0	57.2	5.8	58.6	56.6	4.6	55.6	55.6	3.2	58.7	57.3	4.0	58.2	56.7	4.4
	zlib	50.8	53.1	2.0	51.0	53.3	2.0	47.0	50.5	1.4	49.8	52.4	2.0	49.6	52.3	1.9
	Min-10%	61.3	57.8	5.8	59.9	58.0	4.2	55.7	56.5	3.2	60.5	58.5	4.0	59.3	57.7	4.3
	Min-20%	60.7	58.1	5.6	59.1	57.4	4.6	55.6	55.9	3.2	59.5	57.5	4.0	58.7	57.2	4.3
	Min-10%++	56.9	55.8	3.0	56.4	55.5	2.8	51.4	52.6	2.2	57.0	56.1	2.4	55.4	55.0	2.6
	Min-20%++	57.4	56.2	3.0	56.9	55.4	2.8	52.3	53.7	2.4	57.8	57.2	2.4	56.1	55.6	2.6
	Recall	66.3	63.2	2.2	62.1	60.2	3.4	63.4	61.4	2.0	61.4	59.6	1.4	63.3	61.1	2.2
	Synprune	62.8	60.4	5.4	61.6	59.9	4.6	64.0	62.6	2.0	65.2	61.9	1.4	63.4	61.2	3.4
	Ours	**75.9**	**70.1**	**12.8**	**74.4**	**68.7**	**7.1**	**73.5**	**68.0**	**5.6**	**71.5**	**66.2**	**6.6**	**74.0**	**68.1**	**7.2**
C++	PPL	47.8	51.2	3.0	45.9	50.8	2.4	46.4	51.1	2.0	45.4	50.3	1.2	46.4	50.8	2.1
	zlib	35.8	50.4	1.6	35.5	50.2	1.0	36.4	50.1	0.8	36.0	50.1	0.4	35.9	50.2	0.9
	Min-10%	49.2	51.4	3.2	47.6	51.1	2.4	47.6	51.0	2.0	46.6	50.4	1.2	47.8	51.0	2.2
	Min-20%	48.3	51.2	3.0	46.5	50.8	2.4	46.7	51.3	2.0	45.6	50.3	1.2	46.8	50.9	2.1
	Min-10%++	48.4	51.5	3.0	50.8	52.1	3.6	48.9	51.4	2.8	48.3	51.0	1.8	49.1	51.5	2.8
	Min-20%++	48.2	51.6	3.0	50.2	52.2	3.6	48.1	51.5	3.0	47.5	50.6	1.8	48.5	51.5	2.8
	Recall	68.7	63.1	**10.4**	68.4	63.0	4.4	68.2	63.7	2.6	67.3	62.4	**5.8**	68.1	63.1	5.8
	Ours	**73.1**	**67.5**	6.6	**72.6**	**66.6**	**6.7**	**72.4**	**67.0**	**5.1**	**72.4**	**67.2**	3.0	**72.7**	**66.7**	**6.5**

Table 6: Detection performance comparison across languages and code transformation based on Magicoder-DS-6.7B

Lang	Method	Original			Type-I			Type-II			Type-III			Macro Average		
		AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%
Python	PPL	66.8	63.4	4.8	64.9	62.4	2.4	60.9	58.3	2.0	60.6	58.3	2.0	63.3	60.6	2.8
	zlib	39.4	52.4	0.8	38.8	53.0	0.4	36.0	52.2	0.4	35.5	51.9	0.2	37.5	52.4	0.5
	Min-10%	67.1	63.9	6.4	65.4	62.6	3.4	61.0	59.0	2.6	60.9	58.4	2.4	63.6	61.0	3.7
	Min-20%	67.2	63.7	5.2	65.3	62.0	2.4	61.0	58.1	2.0	60.7	58.0	1.8	63.6	60.5	2.8
	Min-10%++	67.8	64.6	4.6	56.5	57.1	0.6	56.1	57.1	0.2	50.4	53.2	0.4	57.7	58.0	1.4
	Min-20%++	68.4	65.1	4.4	58.0	57.7	0.6	57.4	56.9	0.2	52.6	53.5	0.4	59.1	58.3	1.4
	Recall	77.6	74.5	3.2	71.8	69.8	1.8	74.1	71.5	0.6	72.9	70.9	0.4	74.1	71.7	1.5
	Synprune	79.2	73.3	4.4	78.6	72.3	5.8	76.2	70.6	3.2	75.3	70.6	1.2	77.3	71.7	3.6
	Ours	**90.2**	**82.7**	**25.8**	**88.8**	**80.3**	**13.5**	**88.3**	**79.8**	**11.2**	**87.2**	**79.6**	**12.2**	**88.7**	**80.6**	**15.9**
C++	PPL	60.0	59.3	0.6	59.0	59.0	0.6	51.4	53.2	0.2	50.5	51.2	0.8	55.2	55.7	0.5
	zlib	40.9	51.4	0.2	40.1	51.2	0.2	35.8	50.8	0.0	36.6	50.7	0.2	38.4	51.0	0.1
	Min-10%	61.6	60.4	0.6	61.2	60.1	0.8	52.7	53.6	0.2	51.8	52.1	0.8	56.8	56.5	0.6
	Min-20%	60.7	59.9	0.6	60.4	60.7	0.6	51.9	53.3	0.2	50.8	51.6	0.8	55.9	56.4	0.5
	Min-10%++	62.6	62.4	0.2	46.3	52.1	0.8	45.9	50.9	0.2	41.7	50.1	0.0	49.1	53.9	0.3
	Min-20%++	63.0	62.1	0.2	47.6	52.1	0.8	46.6	51.2	0.2	42.0	50.1	0.0	49.8	53.9	0.3
	Recall	71.5	69.6	0.2	70.2	68.1	1.0	66.9	65.2	0.4	66.7	65.8	0.2	68.8	67.2	0.5
	Ours	**92.9**	**85.7**	**42.0**	**91.1**	**83.0**	**29.9**	**91.4**	**83.7**	**33.8**	**89.2**	**81.4**	**16.0**	**91.1**	**83.2**	**29.8**

Table 7: Detection performance comparison across languages and code transformation types based on Magicoder-CL-7B

Lang	Method	Original			Type-I			Type-II			Type-III			Macro Average		
		AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%
Python	PPL	73.0	67.7	2.4	69.6	65.4	2.6	64.9	61.9	0.4	65.4	61.7	2.0	68.2	64.2	1.8
	zlib	43.2	54.1	0.2	42.0	54.1	0.4	38.1	53.2	0.2	38.2	52.9	0.0	40.4	53.6	0.2
	Min-10%	74.0	68.7	3.0	71.4	67.0	3.2	66.1	63.4	0.4	66.4	62.3	3.6	69.5	65.3	2.5
	Min-20%	73.8	68.2	2.4	70.5	66.4	2.6	65.6	62.4	0.4	65.8	61.6	2.0	68.9	64.7	1.8
	Min-10%++	79.7	74.8	9.8	73.8	68.6	5.4	67.0	64.6	3.2	61.9	59.9	2.6	70.6	67.0	5.2
	Min-20%++	80.5	74.5	10.0	74.7	69.2	5.4	68.5	65.2	3.2	63.8	60.9	3.6	71.9	67.5	5.5
	Recall	82.0	76.8	7.0	81.1	74.5	4.0	78.4	74.6	1.6	76.4	73.4	2.6	79.5	74.8	3.8
	Synprune	84.7	77.5	10.6	83.5	76.8	11.0	78.1	72.2	2.0	77.7	70.9	6.6	81.0	74.3	7.6
	Ours	**91.9**	**85.4**	**27.2**	**90.2**	**82.8**	**19.8**	**88.8**	**82.2**	**20.6**	**87.8**	**81.4**	**16.6**	**90.0**	**83.0**	**21.1**
C++	PPL	65.0	63.4	0.2	61.8	60.2	0.0	55.7	55.4	0.0	52.3	53.6	0.4	58.7	58.1	0.1
	zlib	46.3	51.4	0.0	43.6	50.8	0.0	40.4	51.1	0.0	39.5	50.5	0.2	42.4	50.9	0.1
	Min-10%	66.5	64.4	0.2	63.7	61.7	0.0	54.7	55.0	0.2	53.2	53.3	0.4	59.5	58.6	0.2
	Min-20%	66.3	63.7	0.2	63.0	61.3	0.0	56.3	55.8	0.0	52.6	53.2	0.4	59.5	58.5	0.1
	Min-10%++	73.7	70.0	0.6	58.2	59.3	0.2	52.6	53.2	0.0	46.5	50.9	0.0	57.7	58.4	0.2
	Min-20%++	73.8	70.0	0.6	59.1	60.3	0.2	54.7	55.4	0.0	47.1	50.7	0.0	58.7	59.1	0.2
	Recall	77.3	74.0	0.4	67.4	66.5	0.0	65.2	62.7	0.0	65.3	63.7	0.0	68.8	66.7	0.1
	Ours	**92.9**	**85.5**	**42.4**	**88.6**	**81.3**	**17.0**	**87.7**	**80.5**	**14.8**	**86.7**	**79.2**	**11.4**	**89.8**	**82.2**	**21.0**

Table 8: Detection performance comparison across languages and code transformation types based on octogeex

Lang	Method	Original			Type-I			Type-II			Type-III			Macro Average		
		AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%
Python	PPL	39.5	50.5	0.2	38.3	50.5	0.0	42.3	50.5	0.0	41.0	50.1	0.0	40.3	50.4	0.1
	zlib	27.1	51.5	0.0	27.7	51.2	0.0	33.8	54.4	0.0	30.3	52.4	0.0	29.7	52.4	0.0
	Min-10%	34.5	50.2	0.6	31.9	50.1	0.0	36.0	50.4	0.2	33.1	50.4	0.0	33.9	50.3	0.2
	Min-20%	34.6	50.3	0.0	31.8	50.3	0.0	35.3	50.5	0.0	33.4	50.2	0.0	33.8	50.3	0.0
	Min-10%++	50.6	53.4	1.4	46.7	50.6	0.2	51.4	52.7	0.6	46.3	50.2	0.2	48.8	51.7	0.6
	Min-20%++	51.8	54.0	1.2	47.4	51.1	0.2	52.9	53.9	0.6	48.0	50.8	0.4	50.0	52.4	0.6
	Recall	48.4	53.1	0.0	48.7	52.5	0.2	53.0	55.2	0.0	50.1	53.6	0.0	50.0	53.6	0.1
	Synprune	71.0	67.2	**7.4**	68.0	65.3	**2.8**	66.6	63.3	**3.6**	68.6	64.4	**3.4**	68.5	65.1	**4.3**
	Ours	**80.0**	**73.8**	5.8	**79.0**	**73.0**	2.6	**79.1**	**73.2**	2.4	**76.6**	**71.5**	2.6	**78.7**	**72.6**	3.9
C++	PPL	41.0	51.2	0.0	39.1	50.6	0.0	41.2	51.6	0.0	38.1	50.8	0.0	39.9	51.1	0.0
	zlib	42.4	51.1	0.0	43.9	50.8	0.0	43.3	51.1	0.0	44.3	52.4	0.0	43.5	51.4	0.0
	Min-10%	44.0	50.2	0.4	41.5	50.1	0.0	43.7	50.1	0.0	39.9	50.2	0.0	42.3	50.2	0.1
	Min-20%	43.3	50.0	0.0	40.3	50.1	0.0	42.3	50.1	0.0	39.0	50.6	0.0	41.2	50.2	0.0
	Min-10%++	56.2	56.0	0.2	54.3	54.8	0.4	55.1	56.3	0.4	52.5	54.3	1.2	54.5	55.4	0.5
	Min-20%++	57.4	57.3	0.0	55.5	55.5	0.6	55.9	56.8	0.4	52.8	54.6	0.4	55.4	56.0	0.4
	Recall	51.4	52.2	1.2	48.4	51.3	0.4	49.4	50.8	1.4	48.1	51.1	1.6	49.3	51.3	1.1
	Ours	**70.8**	**65.7**	**3.2**	**70.9**	**65.6**	**4.8**	**71.3**	**66.3**	**5.2**	**67.7**	**63.2**	**4.0**	**70.0**	**64.7**	**4.2**

Table 9: Detection performance comparison across languages and code transformation types based on octocoder

Lang	Method	Original			Type-I			Type-II			Type-III			Macro Average		
		AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%
Python	PPL	46.7	52.6	0.6	43.7	52.6	0.2	48.4	52.7	0.2	45.0	53.5	0.2	45.9	52.8	0.3
	zlib	31.3	52.6	0.0	31.5	53.0	0.0	38.3	56.4	0.0	33.4	54.0	0.0	33.6	54.0	0.0
	Min-10%	49.9	54.8	0.4	47.0	54.0	0.2	51.9	55.9	0.2	48.8	54.4	0.2	49.4	54.8	0.2
	Min-20%	48.1	53.8	0.6	45.2	53.5	0.2	50.3	54.6	0.2	46.4	53.7	0.2	47.5	53.9	0.3
	Min-10%++	46.5	51.5	0.4	46.9	51.5	0.4	49.1	53.0	0.4	47.2	50.6	0.8	47.4	51.6	0.5
	Min-20%++	46.8	52.5	0.4	46.2	52.1	0.4	48.5	53.5	0.4	46.7	51.2	0.8	47.0	52.3	0.5
	Recall	75.0	72.0	1.2	72.9	69.8	0.2	67.1	68.5	0.0	72.1	70.9	0.0	71.8	70.3	0.4
	Synprune	70.8	65.6	2.2	67.2	62.9	1.6	66.1	63.9	0.4	70.6	67.0	1.4	68.7	64.8	1.4
	Ours	**86.9**	**79.5**	**8.8**	**85.7**	**78.5**	**9.7**	**85.7**	**78.8**	**6.4**	**84.0**	**76.9**	**11.2**	**85.6**	**78.0**	**10.0**
C++	PPL	47.6	52.3	0.0	47.8	52.2	0.0	48.5	52.6	0.0	47.4	52.2	0.0	47.8	52.3	0.0
	zlib	44.8	52.9	0.0	46.0	52.7	0.0	46.0	52.8	0.0	45.0	52.4	0.0	45.5	52.7	0.0
	Min-10%	51.0	54.7	0.0	51.4	55.3	0.0	51.2	55.4	0.0	50.6	55.6	0.0	51.0	55.2	0.0
	Min-20%	49.2	54.3	0.0	49.6	54.2	0.0	49.8	54.4	0.0	48.7	53.4	0.0	49.3	54.1	0.0
	Min-10%++	45.6	51.1	0.0	47.3	52.5	0.0	45.3	50.7	0.0	45.8	51.9	0.0	46.0	51.5	0.0
	Min-20%++	46.0	51.4	0.0	47.6	52.5	0.0	45.5	50.8	0.0	46.0	51.5	0.0	46.3	51.6	0.0
	Recall	66.5	65.8	0.4	61.6	60.3	0.4	64.4	62.3	0.6	63.0	62.8	0.2	63.9	62.8	0.4
	Ours	**77.9**	**72.3**	**6.6**	**77.3**	**70.7**	**6.7**	**77.8**	**70.6**	**5.6**	**76.4**	**69.5**	**5.8**	**77.2**	**70.5**	**6.5**

Table 10: Detection performance comparison across languages and code transformation types based on OpenCoder-1.5-Instruct

Lang	Method	Original			Type-I			Type-II			Type-III			Macro Average		
		AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%
Python	PPL	88.3	82.4	8.4	86.6	80.4	20.6	81.2	75.7	5.4	78.6	72.2	7.2	83.7	77.7	10.4
	zlib	76.6	72.3	5.4	74.7	70.5	4.2	68.4	66.2	5.2	65.3	63.1	**11.2**	71.3	68.0	6.5
	Min-10%	88.9	83.0	7.0	86.7	80.6	**24.0**	80.8	75.3	12.0	71.6	68.2	2.6	82.0	76.8	11.4
	Min-20%	88.9	83.0	6.2	86.9	81.3	20.6	81.0	75.8	6.4	71.7	67.1	2.2	82.1	76.8	8.8
	Min-10%++	74.0	70.3	11.8	65.5	63.5	7.0	58.5	59.1	2.6	55.7	55.7	1.6	63.4	62.2	5.8
	Min-20%++	78.3	73.7	13.0	71.4	68.7	7.0	65.1	62.9	2.6	60.0	59.4	1.6	68.7	66.2	6.1
	Recall	66.4	64.0	1.0	58.1	57.9	1.6	59.5	59.4	0.2	60.4	59.6	0.8	61.1	60.2	0.9
	Synprune	86.9	82.2	4.4	85.3	79.5	7.0	79.4	74.2	1.2	77.7	73.5	1.8	82.3	77.3	3.6
	Ours	**93.9**	**87.0**	**49.4**	**90.0**	**83.7**	23.5	**89.7**	**82.7**	**22.4**	**87.6**	**80.8**	9.6	**90.5**	**83.6**	**23.6**
C++	PPL	78.7	74.2	10.8	75.3	70.8	3.2	72.2	67.9	4.8	70.2	66.2	2.2	74.1	69.8	5.2
	zlib	51.5	55.1	0.0	49.5	53.9	0.2	48.5	55.0	0.0	48.8	55.2	0.0	49.6	54.8	0.1
	Min-10%	80.4	75.8	13.8	78.8	73.9	5.4	74.2	69.2	5.6	72.5	68.5	3.0	76.5	71.9	7.0
	Min-20%	79.6	75.0	11.0	77.1	71.9	3.2	73.1	68.3	4.4	71.1	67.8	2.2	75.2	70.8	5.2
	Min-10%++	70.9	69.2	7.4	53.2	57.5	3.0	55.9	56.6	1.8	50.0	53.1	1.2	57.5	59.1	3.4
	Min-20%++	72.5	70.0	7.6	56.6	59.4	3.0	59.5	58.4	2.0	53.8	55.2	1.2	60.6	60.8	3.5
	Recall	67.3	65.1	4.6	64.3	63.5	0.0	59.2	60.2	0.4	57.2	58.3	1.0	62.0	61.8	1.5
	Ours	**96.1**	**89.9**	**54.0**	**94.2**	**87.3**	**34.9**	**93.9**	**86.9**	**33.5**	**92.8**	**85.8**	**29.2**	**94.7**	**87.9**	**38.3**

Table 11: Detection performance comparison across languages and code transformation types based on OpenCoder-8B-Instruct

Lang	Method	Original			Type-I			Type-II			Type-III			Macro Average		
		AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%	AUC	ACC	TPR @1%
Python	PPL	89.5	84.4	3.6	88.3	82.5	14.4	82.3	75.7	1.2	80.7	74.7	4.2	85.2	79.3	5.8
	zlib	77.5	73.0	9.0	75.8	72.0	7.2	68.8	67.4	2.6	67.7	65.7	4.0	72.4	69.5	5.7
	Min-10%	91.2	86.4	5.2	89.5	**83.4**	27.6	83.0	77.9	1.4	81.5	76.2	7.0	86.3	81.0	10.3
	Min-20%	90.2	85.2	3.8	89.0	83.2	16.8	82.6	76.6	1.2	81.2	75.7	5.0	85.8	80.2	6.7
	Min-10%++	84.4	79.2	14.6	73.8	69.3	9.8	63.8	61.7	0.4	60.1	60.5	3.2	70.5	67.7	7.0
	Min-20%++	85.9	80.0	14.8	77.8	72.8	9.8	69.8	65.2	0.4	66.7	63.8	2.8	75.1	70.5	7.0
	Recall	73.4	71.1	1.0	63.9	61.4	0.0	62.6	61.0	0.4	62.4	60.9	0.4	65.6	63.6	0.5
	Synprune	89.4	84.5	4.6	89.1	82.7	0.8	82.3	76.9	0.4	81.3	76.7	1.8	85.5	80.2	1.9
	Ours	**95.3**	**88.2**	**61.4**	**91.2**	83.0	**32.3**	**90.6**	**82.5**	**34.8**	**87.8**	**80.9**	**20.2**	**91.5**	**83.4**	**36.2**
C++	PPL	76.9	71.6	3.2	72.8	68.9	2.0	70.0	66.4	0.2	68.4	64.2	1.2	72.0	67.8	1.7
	zlib	50.2	52.8	0.0	48.0	51.8	0.4	46.7	51.6	0.2	47.0	52.3	0.2	48.0	52.1	0.2
	Min-10%	78.9	73.1	4.8	75.8	71.8	3.0	72.4	67.3	0.4	71.5	66.9	1.0	74.7	69.8	2.3
	Min-20%	78.1	72.7	3.2	74.5	70.1	2.0	71.2	66.9	0.2	69.6	64.8	1.2	73.3	68.6	1.7
	Min-10%++	77.0	72.2	13.6	58.6	60.4	3.0	59.9	60.5	1.6	54.1	56.0	1.6	62.4	62.3	5.0
	Min-20%++	77.0	72.3	13.8	60.7	60.4	3.0	61.3	61.8	1.6	56.2	56.5	1.8	63.8	62.7	5.1
	Recall	80.9	73.8	6.8	76.8	72.5	1.2	74.7	70.4	1.8	73.0	69.6	1.6	76.4	71.5	2.9
	Ours	**94.9**	**87.4**	**50.0**	**92.8**	**85.9**	**34.8**	**92.9**	**85.9**	**35.8**	**90.5**	**81.3**	**19.8**	**92.8**	**84.7**	**35.8**

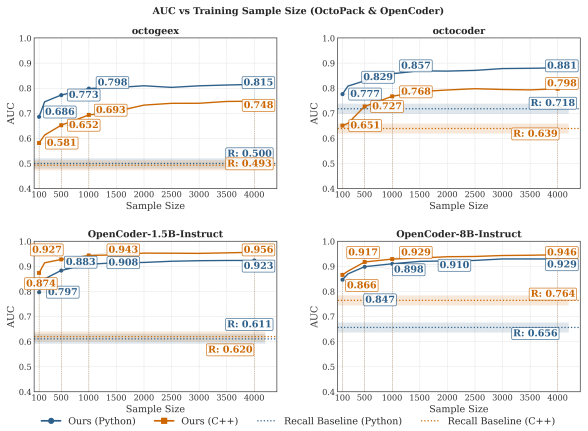


Figure 16: Impact of training data size on detection AUC for the LightGBM classifier, under Octopack and OpenCoder

ing samples are used. With 1K or more training samples, our method outperforms the Recall baseline by approximately 20% on average in AUC. The AUC increases steadily with growing sample size, rising rapidly from 100 to 1K and continuing to climb, though at a slower pace, from 1K to 4K.

C.7 Effect of Classifier Architecture

To investigate the impact of classifier architecture on training data detection accuracy and efficiency, we evaluate six models: L1-regularized logistic regression (L1 LogReg), multilayer perceptron (MLP), RBF kernel SVM, Random Forest, XGBoost, and LightGBM, using 1K training and 4K test samples on CPU, as summarized in Table 12. LightGBM achieves the highest AUC, reaching

Table 12: Classifier performance on Python and C++.

Classifier	Python			C++		
	AUC	Train	Infer	AUC	Train	Infer
L1 LogReg	73.3	2.957	0.013	72.6	4.355	0.013
MLP	74.0	3.657	0.034	72.9	2.595	0.037
SVM (RBF)	74.5	0.380	0.461	74.2	0.392	0.349
Random Forest	82.9	0.860	0.259	81.7	0.652	0.212
XGBoost	82.3	105.083	0.037	81.0	84.928	0.037
LightGBM	84.2	365.062	0.023	82.1	281.381	0.022

LogReg: Logistic Regression; MLP: Multi-Layer Perceptron.
AUC in %; train and inference time in seconds.

84.2% on Python and 82.1% on C++, followed by Random Forest and XGBoost. These ensemble models substantially outperform linear and kernel methods, confirming that membership inference requires highly nonlinear decision boundaries. AUCs across Python and C++ implementations differ by at most 2.1 percentage points. Regarding efficiency, SVM trains fastest but infers slowest due to kernel evaluations, while L1 LogReg is extremely fast in both training and inference, suiting low-latency online scenarios. LightGBM incurs the longest training time, 365s on Python and 281s on C++, yet delivers highly efficient inference, with times of 0.023s and 0.022s, advantageous for the "train once, infer many" deployment pattern. Overall, when prioritizing AUC and inference time, LightGBM is the best choice, as the training time remains acceptable given the small sample size, with XGBoost and Random Forest as strong alternatives.

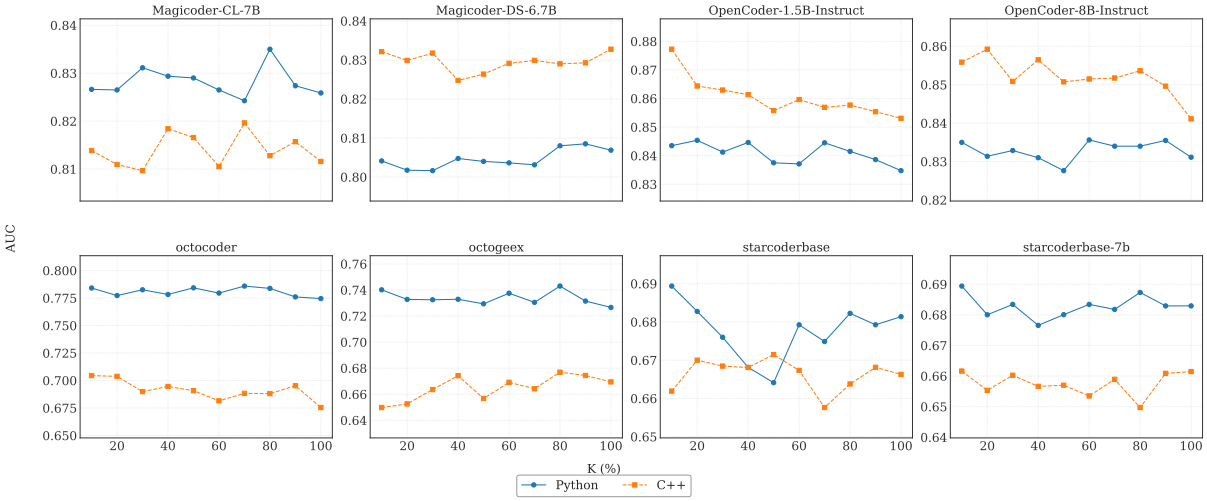


Figure 17: Effect of the hyperparameter K on LightGBM test AUC

Table 13: AUC of different code transformation types, with macro-average.

Lang.	Method	Orig.	I	II	III	Avg.
Python	Combine	86.4	84.4	83.9	82.1	84.2
	Log Prob	85.4	83.4	83.1	81.5	83.3
	Norm Log Prob	84.1	81.5	80.8	78.3	81.2
C++	Combine	83.8	82.0	81.8	80.5	82.0
	Log Prob	82.5	81.1	80.8	79.3	80.9
	Norm Log Prob	81.9	80.0	79.9	78.5	80.1

AUC values are reported as percentages ($\times 100$) without the % symbol. "Avg." denotes the macro-average over clone types.

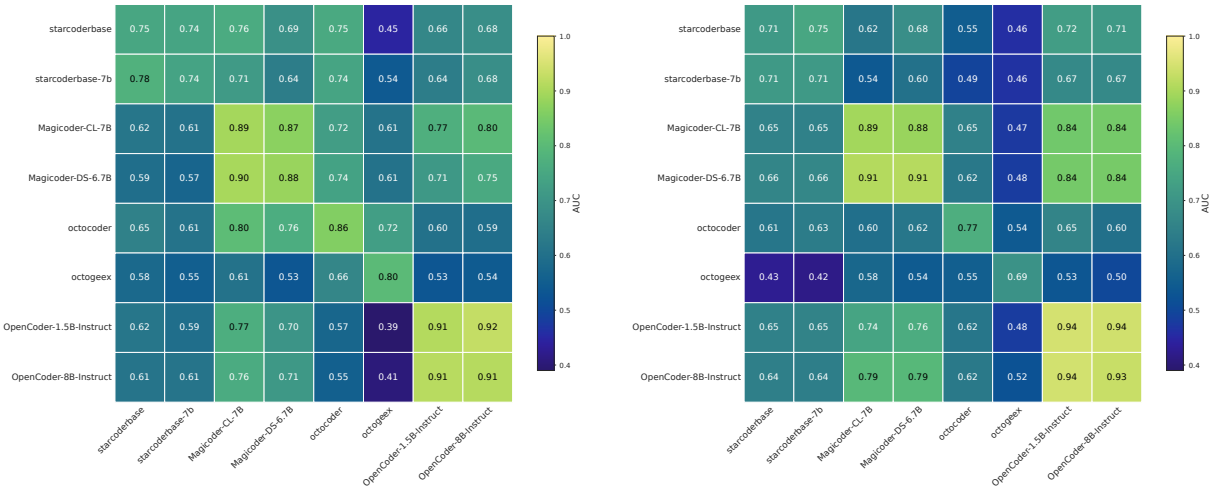
C.8 Effect of Logp. vs. Norm Logp.

As shown in Table 13, we summarize two main findings. **(1)** Across all code transformation types, concatenating the raw log probability $\ell(x_i)$ and the z-score normalized log probability $\hat{\ell}(x_i)$ as features for the LightGBM classifier consistently yields higher membership inference AUC than using either feature alone, confirming that richer supervision benefits discrimination. **(2)** More surprisingly, using only $\ell(x_i)$ outperforms using only $\hat{\ell}(x_i)$. We attribute this gap to how z-score normalization compresses absolute-scale discrepancies per prefix. Consider a concrete per-prefix distribution of log probabilities over the vocabulary: for a fixed context $x_{<i}$, the model outputs raw values $[-10,\ 0,\ 0,\ 0,\ 0]$ for five candidate tokens. The mean μ of these log probabilities is -2 and the standard deviation σ is 4; after normalization, the vector becomes roughly $[-2,\ 0.5,\ 0.5,\ 0.5,\ 0.5]$. The original extreme value -10 encodes substantial model surprise—such sharply low log probabilities are more characteristic of non-member code, where the model has not over-fitted to that prefix. Z-score normalization pulls this outlier toward the mean, smoothing away the magnitude that discriminates members from non-members and thereby weakening the signal. When both $\ell(x_i)$ and $\hat{\ell}(x_i)$ are provided jointly, the classifier can exploit the absolute confidence scale together with the relative vocabulary ranking, which explains the improved AUC.

C.9 Effect of Hyperparameter K

K specifies the percentage of lowest-probability tokens retained per programming-language (PL) syntactic type. For each type, we compute the mean log-probability and mean normalized log-probability over the Min-$K\%$ tokens, yielding per-type scalar features. The concatenated feature vector across all syntactic types is then used to train and evaluate the LightGBM classifier.

As shown in Figure 17, for most models (e.g., *OpenCoder-1.5B-Instruct*, *octocoder*, and *starcoderbase*), the AUC of a LightGBM classifier trained on the mean confidence of the Min-$K\%$ tokens within each syntactic category generally decreases as K increases. For instance, on C++ code, the AUC of *OpenCoder-1.5B-Instruct* drops by approximately 2.5 percentage points as K grows from 10% to 100%. This suggests that low-confidence tokens are more discriminative for contamination detection: in contaminated samples, models exhibit overconfidence on tokens that are typically hard to predict, producing abnormally high confidence scores that reveal contamination traces, consistent with Shi et al. (2023); Zhang et al. (2025). A few models, such as Magicoder-CL-7B, do not show a clear downward trend, likely due to model-specific confidence calibration and the stochasticity of test-set sample selection.



(a) Python (b) C++

Figure 18: Generalization across different models

C.10 Generalization Across Different LLMs

In this section, we examine the generalization of our training data detection method: classifiers are trained on token log probabilities and normalized token log probabilities from the dataset of one LLM, and tested on the dataset of another LLM. Results are shown in Figure 18, with the training model on the vertical axis and the evaluation model on the horizontal axis. We observe that generalization is consistently better within the same model family (the near-diagonal region). For example, a classifier built using the training data of *Magicoder-DS-6.7B* attains an AUC of 0.90 when evaluated on the training data of *Magicoder-CL-7B*; similar behavior holds for *OpenCoder*, and for the pretrained *StarCoder* family the cross-model AUC remains around 0.75. In contrast, generalization across different families is markedly weaker, with AUCs ranging from 0.6 to 0.8. This drop arises because different model families, due to distinct hyperparameters such as training epochs and optimizers, yield inconsistent token confidence scores, which degrades classifier performance. In the same family, however, despite differences in model size and architecture, the shared training paradigm keeps token confidence largely consistent, preserving high cross-model AUCs.